

# IndusOPT: An Industrial-Scale Benchmark for Natural-Language Optimization Modeling Built Forward from Real Production Scenarios

Fan Zhang Kai Wang Junyan Liu Zhaoyi Sun Xiwen Wang Qi Chen  
Xumeng Cao Liang Qin Yingying Cheng Roscoe Wang Jie Sun

[Camera Endorser] [Communication Endorser] [HarmonyOS Endorser] [CAE/Computing Endorser]  
[Compiler Endorser] [Digital Power Endorser] [DataCom Endorser] [Optical Network Endorser]\*

Huawei Technologies

## Abstract

We present IndusOPT, a benchmark of 17 optimization problems and 128 data instances built *forward* from real production scenarios at Huawei, spanning camera pipelines, operating-system scheduling, sparse linear algebra, LLM inference placement, datacom and optical networks, and energy systems. Each problem is written in Chinese by the engineer who owns the scenario and ships a machine-readable provenance and desensitization record, a reference formulation, a reference Pyomo implementation, and audited reference solutions. Prose is separated from data: each problem provides 5–15 instances spanning 33 to 5,310,086 variables-plus-constraints across MILP, IP, LP, QP, and MINLP, so a submission must bind data rather than reproduce a worked example. This is four orders of magnitude beyond the textbook corpora that still dominate the field, on data that has never been public.

Measurement receives the same engineering as data. Evaluation is closed-book: generated models are executed by us under a version-pinned solve regime and judged through four gates, namely solvability, feasibility re-verification, bidirectional cross-validation against the reference model, and objective match at  $10^{-6}$ . Two audits then measure the benchmark itself. A drop-one constraint-coverage audit asks whether each problem can detect an omitted constraint at all, and publishes the blind spots it finds, including one family we show to be provably undetectable. An audit of the judge caught it being wrong once, and the archived runs were re-judged offline.

Across 68 closed-book runs, a frontier coding model inside an execution-capable agent harness passes 82.3% of judgeable instance-runs without annotations, our primary condition. We report this plainly rather than claiming the corpus defeats current systems, because what makes it worth releasing is that it can decompose the number. The judgeable set is biased toward what is easy to *verify*, so we report the conservative denominator alongside. The expert-written implicit-constraint annotations, which are a document no deployment has at first contact, are worth up to a whole problem, though repeated runs show that the sharpest instance of that effect was partly a statement defect we have since repaired, and we report the withdrawal. And a pass certifies less than it appears to: the cross-check confirms model agreement on only 42 of 512 instance-runs and abstains on 401, so counting only confirmed passes the rate is 5.3%. Failures concentrate in trace-driven world-modeling, MINLP, and large-scale data binding, all of them regimes that textbook corpora cannot expose, and 21 of 128 instances carry data too large to inline into a 128k-token prompt at all, where the field’s current self-contained-prompt interface does not apply.

---

\*Author list and ordering to be finalized. Lines 1–2: the current scenario submitters. Lines 3–4: placeholder slots for the product-line endorser, who join as co-authors per the repository’s endorsement policy once they confirm desensitization and approve publication.

# 1 Introduction

Turning a business requirement into a correct mathematical program is the oldest bottleneck in operations research, and large language models now write plausible optimization code from plain prose. Measuring whether that code is *right* has become its own research problem. The measurement instruments, however, lag the claim: the benchmarks that dominate the literature are textbook word problems of a few dozen variables, judged by comparing a single number. On such corpora, frontier systems exceed 85–95% accuracy [16, 29], and recent work shows the same systems collapsing to single-digit pass rates the moment problems acquire industrial scale and structure [22].

A field whose instruments report either saturation or collapse cannot tell which of its components is responsible for either. That is the situation this paper addresses. Three gaps separate the instruments from the reality they claim to measure.

**The realism gap.** Existing corpora are predominantly synthesized, whether by templates, by LLMs, or by paraphrasing textbook exercises. Even IndustryOR [16], which claims industrial origin, ships abstracted miniatures averaging tens of variables. The strongest recent counterpoint, MIPLIB-NL [22], obtains real industrial *structure* by construction in reverse: it starts from anonymized MIPLIB 2017 instances and has experts hypothesize a business story to wrap around the mathematics. The narrative realism of such text is bounded by the reconstruction: the original requirement, its terminology, and its implicit conventions are gone.

**The scale gap.** A model that binds three scalar parameters into a three-variable LP exercises none of the machinery that industrial modeling lives on, such as index discipline, data binding, and constraint families instantiated in loops. Benchmarks whose “hard” tier stops at hundreds of variables saturate, and saturation gets misread as progress.

**The verification gap.** Scalar objective comparison is a partial oracle: it passes models that omit constraints yet land on the right number, and several widely used answer keys have audited error rates of 20–54% [22]. The literature’s own error analyses point the same direction, since omitted constraints are the dominant failure mode of trained modelers [16], yet the standard judge cannot see them.

**This work.** We present IndusOPT, a benchmark of 17 optimization problems and 128 data instances built *forward* from real production scenarios at Huawei, spanning smartphone camera pipelines, HarmonyOS kernel scheduling, sparse linear-algebra kernels, LLM inference placement, datacenter and optical networks, microgrids, and virtual power plants. Each problem is authored in Chinese by the engineer who owns the production system, carries a machine-readable provenance and desensitization record, and ships five verifiable artifacts: the natural-language statement, structured instance data, a reference formulation, a reference Pyomo implementation, and audited reference solutions. Instances span 33 to 5,310,086 variables-plus-constraints, which reaches from interview-question size to beyond the largest published NL-to-optimization instances, and they cover MILP, IP, LP, QP, and MINLP.

Measurement receives the same engineering as data. Our closed-book protocol gives the model only the statement, the data dictionary, and one sample instance, then runs the generated `build(data)` ourselves under a pinned, version-controlled solve regime. Verdicts pass a four-gate judge: solve, re-verify feasibility on the generated model, substitute the solution bidirectionally against the reference model, and match the objective at  $10^{-6}$ . The judge itself is audited, and one judging bug was caught and repaired through offline re-judgment of archived runs. A constraint-coverage audit asks the question benchmarks usually skip: for every constraint family, does at least one instance actually move when it is dropped? Where the answer is no, we publish the blind spot.

**What we find, and why a high score is the point.** On the current release, strong commercial coding models pass 85.0% (claude-opus-5) and 74.3% (composer-2.5) of judgeable instance-runs. We state this plainly rather than framing IndusOPT as a corpus that defeats current systems, because the honest reading is more useful than either extreme. A capable model inside an execution-capable agent harness genuinely does solve most of a forward-built industrial corpus. That is a substantial change from what reverse-constructed industrial instances of comparable scale report, where nine bare LLMs average 26.9% pass@1 (range 21.4%–39.1%) and fine-tuned specialists sit at or below 5.5% [22], but the comparison has to be read with its mechanism attached. None of those systems ran inside an execution-capable harness, and that corpus reports an execution-level failure rate of 47.6%: on its authors’ own reading, most attempts there fail before semantic correctness can be assessed at all. An agent loop that reads tracebacks and revises is built to absorb exactly that class of failure, so the gap between the two numbers is, to an unknown extent, a gap between harnesses rather than between models. Deciding how much of it is which is the attribution question below.

What makes the benchmark worth releasing is that it can say what that number is made of, and each layer it separates argues against reading it as solved. *The denominator is biased*: the 15 instances excluded as unjudgeable are those whose reference solutions open solvers could not certify, and they come from the largest problems, so the judgeable set is skewed toward what is easy to verify. On all 128 instances the same runs give 75.0%, which we report alongside. *Part of the score is purchased with information a deployment does not have*: the annotation ablation removes an expert-written record of implicit constraints, a document that does not exist when a requirement is first handed over, and it moves whole problems, in both directions, since on one the document actively misleads one model and not the other. It also moved one of our own conclusions: repeated runs (Section 6.8) show the sharpest annotation effect we reported was partly a defect in the statement that we have since repaired, and we withdraw that claim rather than leave it standing. *And a pass certifies less than it appears to*: across 512 instance-runs the bidirectional cross-check confirms model agreement on 42 and abstains on 401, so most passes attest objective agreement and feasibility rather than structural equivalence. Counting only confirmed passes, the same runs give 5.3%. The coverage audit also exhibits one constraint family that no instance can detect, meaning a model omitting it outright scores perfectly under either reading. Failures concentrate in trace-driven world-modeling, MINLP, and large-scale data binding, and 21 instances carry data too large to inline into a 128k-token prompt, a regime in which the self-contained-prompt interface assumed by current specialist models does not apply at all.

The open question this benchmark exists to answer is therefore attribution: how much of that rate belongs to the model, how much to the agent loop that executes and revises, and how much to the annotation file. A corpus on which every system scores near zero cannot decompose anything, whereas one with headroom on both sides can. We isolate the annotation layer here and leave the other two open, for a reason we state rather than defer: the evaluation runs through an agent SDK with no harness-free completion path, so the control this needs could not be run (Section 6.9).

## Contributions.

1. **A forward-built industrial benchmark**: 17 problems / 128 instances from real Huawei production scenarios, in Chinese, with named authorship, expert endorsement, explicit per-problem provenance (real/hybrid/synthetic) and an auditable desensitization protocol, scaling to 5,310,086 variables-plus-constraints and five model classes, on data never previously public.
2. **A layered, self-auditing evaluation design**: closed-book sandboxed execution with verified isolation, and a four-gate judge with bidirectional reference cross-validation that abstains rather than guesses when two modelings are structurally different. We report it here as a design and its findings, including the judging bug it was caught making, since the toolchain itself is not part of the release (Section 5.5).
3. **Benchmark self-measurement**: a constraint-coverage metric that quantifies what each problem can

and cannot detect, published together with the blind spots it found in our own data.

4. **An empirical study** of two frontier coding models under annotation ablation, reported against two denominators, with a failure taxonomy separating five kinds of zero and a documented case of the judge being wrong and repaired, and, from that, an attribution question (model vs. harness vs. supplied domain knowledge) that the benchmark is built to answer and that a saturated or a collapsed corpus could not pose.

## 2 Related Work

**NL-to-optimization benchmarks.** NL4Opt [32] established the task with 289 test problems of textbook-style linear programming, and most successors inherit its shape: MAMO [17] (652 easy + 211 complex synthetic problems), ComplexOR [40] (a few dozen), OptiBench/ReSocratic [42] (816 and 605 reverse-synthesized problems), and NLP4LP [1] (361 problems, of which 354 are textbook-style and 7 are case studies adapted from published applications with synthetic data). Adjacent evaluation resources probe other facets: ORQA [31] tests OR reasoning as multiple-choice QA rather than end-to-end modeling, an INFOR benchmark grounds answers in executable Zimpl models [30], LogiOR [41] specializes to logistics, EOR [45] probes explainability, and ORGEval [38] contributes a model–data-separated problem set. IndustryOR [16] claims industrial origin for its 100 problems, but they are abstracted, single-paragraph miniatures, since the companion training statistics report average model sizes around 30 variables even in the hard tier, and the answer key is reduced to a scalar optimum. Across this family three gaps recur, and independent audits report answer-key error rates from 8% to 54% on several of these corpora [22]: problems are toy-sized, numeric data is inlined into the prose (so one problem is one instance), and judging reduces to executing generated code and comparing one number. The first gap is starker than the literature’s qualitative framing suggests: we measured every problem in NL4Opt’s test split and NLP4LP’s public easy split from their own reference artifacts, and the *median* model has 5 variables plus constraints (maxima 8 and 10, as shown in Figure 2), while MAMO, IndustryOR, and OptMATH-Bench ship no reference models at all, so their instance sizes cannot even be audited.

Closest to us, and complementary, is MIPLIB-NL [22]: 223 instances built *in reverse* from MIPLIB 2017 [13] by expert-recovering a loop-level structure from anonymized MILP files and generating an English description from it, reaching  $10^6$  variables-plus-constraints (the 216 we could rebuild from its released artifacts top out at  $5.9 \times 10^5$ ) and exposing a collapse of existing systems: nine bare LLMs average 26.9% pass@1 (21.4%–39.1%) and five fine-tuned specialists score at or below 5.5%, against 17.9% averaged over every system it evaluates. Its own diagnosis is worth carrying forward: an execution-level failure rate of 47.6% means most of that collapse happens before a model’s semantics can be judged. IndusOPT agrees with its diagnosis that scale and structure are the bottleneck, and supplies what reverse construction cannot: original business semantics written forward by the owning engineers, Chinese working-language statements, multiple data instances per scenario (128 over 17 problems, up to 5,310,086 variables-plus-constraints, and model classes beyond pure MILP), on data that has never been public and therefore cannot sit in any pretraining corpus. Where MIPLIB-NL validates that its generated text *suffices* to reconstruct the model, IndusOPT additionally audits whether its instances *discriminate* (constraint coverage) and whether its judge itself is sound (Section 5.3).

**Concurrent 2026 benchmarks, and what is actually unclaimed.** Three concurrent efforts move in the same direction and bound our novelty claim, so we state the boundary rather than the slogan. Opti-Agent-Bench [12] evaluates agents over the whole optimization R&D pipeline of understanding, formulation, implementation and reporting, on business-language tasks with anti-template traps, and checks consistency *across an agent’s own modules*, catching constraint omission and model–code divergence. That cross-module check is internal

to the submission, whereas IndusOPT’s cross-validation is against an independent reference model, and neither subsumes the other. OPT-Engine [8] takes the orthogonal route of procedurally scaling ten canonical OR problems to arbitrary complexity, which buys controlled difficulty at the cost of business semantics, the opposite trade to ours. MM-OptBench [21] extends solver-grounded evaluation to multimodal inputs. In the constraint-programming tradition, IndusCP shares both our diagnosis and part of our design: 140 industrial-level tasks, each with 2–5 test cases at different scales, which is the same model–data separation argument arrived at independently [34]. Alongside these, recent systems work localizes formulation errors [27] and auto-formulates large models few-shot [23].

We therefore do not claim to be the first industrial optimization benchmark. IndustryOR claimed that in 2024. MIPLIB-NL, Opti-Agent-Bench and IndusCP all have strong claims on the territory, and multiple instances per problem is not ours alone. What is unclaimed, to our knowledge, is the specific combination IndusOPT occupies. Problems are written *forward* by the engineers who own the production systems, rather than reverse-constructed from solver files or authored by a benchmark team. Machine-readable provenance and desensitization records state per problem how real the data is. Statements stay in the engineers’ working language rather than being translated into benchmark English. And the benchmark’s own discriminative power is audited and published, blind spots included. Any one of these exists elsewhere. We are not aware of a corpus that has all of them, and the last is the one we would most like others to adopt regardless of what they think of the rest.

**Systems and trained modelers.** A second line builds solvers of the task rather than measurements of it. It covers agentic pipelines such as OptiMUS [1, 2] and Chain-of-Experts [40], with generic self-reflection frameworks [35] as a common ancestor, fine-tuned models such as ORLM [16] and LLMOPT [19], fed by increasingly careful synthetic-data pipelines [24, 28, 39, 46], solver-informed and test-time reinforcement learning such as PEARL [29], SIRL [7], and OR-R1 [10], and finally structured prompting and full modeling agents such as OptiTree [26] and MM-Agent [25]. Their own findings motivate a benchmark like ours. ORLM’s error analysis attributes 56% of modeling failures to incomplete models, that is, to omitted constraints that scalar-objective judging cannot detect. OptiMUS reports that a fine-tuned specialist scoring 73% on its hard split drops to 0/7 on real case studies, and explicitly calls for large, realistic datasets with data files and verified solutions. PEARL’s analysis shows its RL gains come predominantly from repairing implementation errors (74% of repaired failures), while semantic modeling errors dominate the residual, and its limitations section names enterprise-scale instances and the partial-oracle nature of objective-matching as open problems. IndusOPT is one benchmark designed so that those two admissions become measurable: closed-book single-shot evaluation suppresses the execution-repair advantage, and the layered judge targets exactly the semantic errors that survive it.

**Judging protocols.** Most benchmarks above judge by execution accuracy against a stored optimum, while NLP4LP adds a multiset comparison of decision-variable values, which its authors note misjudges legitimate reformulations. Recent work argues that objective matching alone leaves structural hallucinations undetected [20], and proposes graph-theoretic model comparison as an execution-free alternative [38]. IndusOPT’s protocol, which combines feasibility re-verification on the generated model, name-matched bidirectional substitution into the reference model with principled abstention, objective tolerance matched to the solver gap, and offline re-judgment of archived runs when the judge changes, is, to our knowledge, the most heavily audited judging chain in this literature, and Section 6.6 quantifies both what it catches and where it abstains.

## 3 The IndusOPT Benchmark

### 3.1 Design principles

IndusOPT is built *forward*: each problem starts from an optimization need that arose in a real production system at Huawei, is written up by the domain engineer who owns that system, and is then carried through formulation, code, and solution by the same team that understands the business constraints. This is the opposite direction from reverse-constructed benchmarks such as MIPLIB-NL [22], which start from an anonymized solver instance and reconstruct a plausible business story around it. Both directions produce industrial *structure*, but only the forward direction preserves the original business *semantics*, meaning the terminology, the implicit conventions, and the ambiguity profile of a requirement as an engineer would actually state it.

Four principles shape the benchmark:

1. **Problem–instance separation.** A *problem* is a natural-language scenario plus a data dictionary, and an *instance* is a structured data file. Each problem ships 5–15 instances that vary along declared axes (Section 3.5), so a model must generalize over data rather than memorize one worked example. Instance data never appears inside the problem statement.
2. **Every claim is checkable.** Each problem carries a reference formulation, a reference Pyomo [15] implementation, and per-instance reference solutions with solver status, objective, bound, and gap. A toolchain (Section 5.1) verifies each link of the chain from prose to solution, and optimality claims are re-audited by re-solving.
3. **Honest provenance.** Every problem declares its data origin (*real*, *hybrid*, or *synthetic*) and its desensitization record in a machine-readable registration card. We do not claim raw production data where we do not ship it.
4. **Scale is part of the task.** Instances span from 33 to 5,310,086 variables-plus-constraints. At the upper end, naïvely inlining the data into a prompt is impossible and the generated model itself becomes a systems artifact: the benchmark deliberately exposes the data-binding and index-discipline failures that toy-scale corpora cannot.

### 3.2 Problem schema

Each problem is a directory with five artifacts plus a registration card (Table 1). The natural-language statement is written in Chinese, the working language of the engineers who own these systems, as flowing prose without headings, and contains no instance data. A companion `annotations.md` records implicit constraints, domain terminology, and known pitfalls. It is intended for reviewers, and the evaluation protocol treats its inclusion as a controlled ablation switch (Section 5). The data dictionary (`data/README.md`) documents every field of every instance file. The reference formulation gives the mathematical model in  $\text{\LaTeX}$  with one subsection per constraint family ( $C_n$ ), and the reference implementation exposes a single `build(data)` entry point that constructs the model from data with no hard-coded numbers. Reference solutions record complete decision-variable assignments rather than just objective values, which is what makes independent re-verification possible.

### 3.3 One problem, end to end

Figure 1 shows a complete problem at reading size, because the schema above describes a shape and the shape is not the point. The example is the datacom load-balancing problem, chosen because it is the smallest

Table 1: Artifacts of one IndusOPT problem. Files marked \* are per-instance, while the rest are shared by all instances of the problem.

| Artifact            | File                 | Content                                                     |
|---------------------|----------------------|-------------------------------------------------------------|
| Statement           | nl/statement.md      | Business problem, Chinese prose, no instance data           |
| Annotations         | nl/annotations.md    | Implicit constraints, terminology, pitfalls                 |
| Data dictionary     | data/README.md       | Per-file, per-field documentation                           |
| Instance data*      | data/inst_k/         | JSON / CSV / Matrix-Market files                            |
| Formulation         | formulation/model.md | $\text{\LaTeX}$ model, one section per constraint ( $C_n$ ) |
| Reference code      | code/model.py        | Pyomo build(data), solver-agnostic                          |
| Reference solution* | solution/inst_k.json | Full variable assignment, status, objective, bound, gap     |
| Registration        | meta.yaml            | Origin, sensitivity, endorsement, instance axes, sizes      |

in the corpus (17 variables on its first instance) and because it is the benchmark’s cleanest annotation probe (Section 6.5).

Two things about it are worth noticing before the results. The statement is business prose: it names sites, uplink path configurations and service quality, and it never says “epigraph”, “binary variable” or “minimise the maximum”. Recognising that per-node peak congestion is a max that has to be linearised is modelling work the statement deliberately leaves undone (Section 4.1 describes the revision that removed a version which had done it for the reader). And the statement contains no numbers: the importance weights  $w_s$ , the capacities and the eight time periods all live in data/, so the same prose poses eight differently-shaped problems and a submission has to bind data rather than pattern-match a worked example.

#### nl/statement.md: business prose, no data, no formulation

“We operate a hierarchically organised service network. At the bottom are sites with end devices, above them second-level aggregation, then first-level branches, then the core. . . . Planning has to choose an uplink for the services at each site, so that no few links drag the whole network down at peak, without sacrificing service quality more than necessary. . . . Unlike free routing and free splitting, each service must pick one of a set of pre-provisioned **path configurations**.” (*English gloss of a Chinese original.*)

#### formulation/model.md: one subsection per constraint family

$$\min_{x,b,z} \frac{w_1}{|S|} \sum_{s \in S} w_s z_s + \frac{w_2}{|H|} \sum_{h \in H} w_h z_h + \frac{w_3}{|K|} \sum_{k \in K} \sum_{r \in R_k} (1 - g^{(k,r)}) x_r^k$$

(C1)  $\sum_{r \in R_k} x_r^k = 1$  one configuration per service

(C2) link load and utilisation, with hard capacity

(C3)  $z_u \geq b_t^l$  epigraph linearising per-node peak congestion

#### code/model.py: one build(data), every number from data

```
def build(data: dict) -> ConcreteModel:
    m.x = Var(m.KR, domain=Binary)      # configuration choice
    m.z = Var(m.U, domain=NonNegativeReals, bounds=(0.0, 1.0))
    m.C1 = Constraint(...)              # names match (C1)
    m.C2 = Constraint(...)
    m.C3 = Constraint(...)
    return m                            # never solves
```

**solution/inst\_001.json: full assignment, not just a number**

```
{ "status": "optimal", "objective": 0.8293704351332404,  
  "bound": 0.8293704351332404, "gap": 0.0,  
  "solver": " highs 1.15.1", "variables": { ... } }
```

Figure 1: One IndusOPT problem end to end (datacom load balancing, the smallest in the corpus). The statement carries business semantics and no data. The formulation numbers its constraint families so the correspondence to code can be checked mechanically (Section 5.1). `build(data)` reads every number from the instance and never solves, so we control the solver. And the reference solution records the complete variable assignment, which is what makes independent re-verification and the cross-check of Section 5.3 possible at all.

### 3.4 Provenance, desensitization, and endorsement

The 17 problems come from four Huawei business areas: consumer devices (JPEG quantization-table design and video-stabilization trajectory smoothing in smartphone cameras, cell handover on high-speed rail and metro lines, thread scheduling and memory eviction in the HarmonyOS operating system), computing infrastructure (pivot ordering for sparse  $LDL^T$  and LU factorization, cross-pod cluster load balancing, mixture-of-experts expert placement for LLM inference, memory allocation in the TBE AutoFuse compiler), digital power (microgrid sizing-and-operation co-optimization, day-ahead adjustable-load scheduling for virtual power plants), and network infrastructure (datacom traffic load balancing, capacity-expansion planning, and optical-network routing-and-wavelength assignment under link and path protection). Each problem has a named engineer-owner and passed maintainer review before admission, and the registration card carries an endorsement field for product-line experts.

Because the scenarios are real, the data frequently cannot be. Each registration card declares a sensitivity level (15 problems `public`, 2 `internal`) and an explicit record of the desensitization applied, e.g. site names replaced by opaque identifiers with the mapping withheld, or cost vectors rescaled by an undisclosed factor chosen to preserve the optimizer’s structure.

Provenance is a declared three-valued field, and we report it exactly as the cards declare it rather than folding the middle value into the favorable end. 2 problems are `real`: the two sparse-pivoting problems, whose instances are unmodified matrices from the public SuiteSparse collection. 2 are `hybrid`, meaning measured data under an engineer-authored model. These are the JPEG problem, whose rate and distortion tables are computed from the public Kodak image suite, and the HarmonyOS thread problem, whose instances are anonymized production `ftrace` scheduler logs. The remaining 13 are `synthetic`: engineer-authored data laid over the real scenario structure. Only the HarmonyOS traces are production measurements in the strict sense, while a public photographic test set and a public matrix collection are real data but not *our* production data, and the cards say so. The realism claim of IndusOPT rests on scenario provenance, authorship, and the audited desensitization protocol, rather than on an assertion that the numbers are production measurements.

On authorship: every statement, formulation, data file, and reference solution in IndusOPT was written or signed off by the engineer who owns the scenario, and no problem was synthesized from a template or generated wholesale by a language model. We stop short of the stronger claim that no LLM touched any character of the corpus: the repository’s review pipeline is LLM-assisted (Section 5.1), and one round of statement revisions (Section 4.1) was drafted with model assistance before the owning engineer reviewed and merged it. The invariant we do assert is human ownership of every artifact, which is the property the realism claim needs.

### 3.5 Instance design

Each problem declares the axes along which its instances vary, e.g. network size, demand volume, planning-horizon length or volatility regime, with between 2 and 7 axes per problem, and instances are placed to spread those axes rather than to rescale a single dimension. This matters for measurement: on the video-stabilization problem, a systematic modeling error (anchoring crop-corner constraints to the wrong frame geometry) is invisible on six of eight instances and only manifests on the two whose crop margins are tight, so a single-instance benchmark would score the erroneous model as perfect (Section 6).

### 3.6 Statistics

Every statistic in this paper describes one frozen release, IndusOPT v1.0, whose manifest and commit (4d851173f8-dirty) are recorded with the corpus. The repository keeps accepting problems, so a paper that reported “whatever is in the repository” would describe a corpus that no longer exists by the time it is read, and pinning to a manifest is what lets the tables, figures and the evaluation of Section 6 refer to the same 17 problems. Problems admitted after the evaluation sweep are excluded from v1.0 rather than reported without matching results.

IndusOPT v1.0 comprises 17 problems, 15 sub-domains, and 128 instances: 12 MILP, 2 IP, 1 LP, 1 QP, and 1 MINLP problems (Figure 2). Measured as #vars + #cons of the reference model, instances divide into 3 small ( $< 10^2$ ), 95 medium ( $10^2$ – $10^4$ ), 22 large ( $10^4$ – $10^6$ ), and 8 very large ( $\geq 10^6$ ). The largest instance, which is microgrid sizing over a multi-year horizon, has 5,310,086 variables plus constraints, and the mixture-of-experts and virtual-power-plant problems also exceed  $7 \times 10^5$ . Table 4 in the appendix lists every problem with its model class, instance count, and size range, and Appendix D describes each problem’s production context, decision, and difficulty in one paragraph.

## 4 Not Every Submission Deserves a Model and a Solver

A benchmark entry is five artifacts that have to agree with one another. When one of them is wrong, the entry is not merely weaker but actively misleading, in one of two directions: it scores a wrong model as correct, or a correct model as wrong. Both are worse than having no entry at all, and neither announces itself. This section is what the statistics in Section 3 cannot show: the defects the released corpus has been screened against, what each looked like in a real submission, and what the repair was. It is written for the benchmark’s users rather than its imitators, since the point of the exhibits below is not a procedure to copy but a warrant for what ships. A corpus is only as good as the class of errors someone went looking for. Every criterion was learned from a submission that failed it, over 101 reviewed pull requests of which 92 merged. We name the datasets, the repairs are all in the public history, and Table 2 counts every occurrence.

### 4.1 Statements that do not determine the model

The admission test we converged on is a re-derivation test: reading only what the model under test reads, namely statement, annotations, data dictionary and one sample instance, can an independent modeler reconstruct the reference formulation? Where they cannot, the statement is defective, not difficult.

The protocol is worth stating, because “we reviewed the statements” is not a method. All 17 statements were taken in one pass. For each, a modeler with access only to the four inputs above, and explicitly not to `formulation/model.md`, `code/model.py` or `solution/`, wrote a fresh model from the prose, and its optimum was compared against the stored reference objective at the  $10^{-6}$  scoring tolerance. Agreement means the statement determines the model, while disagreement, or an inability to proceed without guessing, localises a specific gap in the prose. The repair direction is fixed in advance and is the part that keeps the test

honest: an ambiguous statement is repaired by changing the *statement*, never the model. *formulation/*, *code/* and *solution/* were untouched throughout, so the reference solutions the evaluation scores against are unaffected by this pass and the before/after comparison below is measured against a fixed key. The pass is archived as a single reviewable change with its per-problem reconstruction results.

Two caveats we would want from someone else’s paper. The reconstruction was performed with model assistance and reviewed by the maintainer, not by an independent human cohort, so it establishes that the statements *determine* their models rather than that a human unfamiliar with the domain would find them easy. And the modeler was not blind to the existence of the reference, only to its content. Of the 17 statements, seven were judged faithful and left untouched, and ten were revised.

**Constants that exist only in the answer key.** The HarmonyOS thread-scheduling problem identifies the main thread by *thread ID* but aggregates candidate roles by *thread name*, which are opposite conventions, neither of them stated. After anonymization dozens of threads share the main thread’s name, so a modeler reading the statement builds a wake-up forest with dozens of spurious roots, and every downstream statistic shifts. The scoring formula’s tie-breaking constant  $\varepsilon = 10^{-6}$  appeared only in the withheld reference formulation, so models guessed  $10^{-3}$  and produced systematically inflated objectives. The revision (Figure 3) states both conventions and the constant explicitly, and the problem went from 0/5 in all four evaluation configurations to 5/5 in all four. The principle the revision established: *a constant that exists only in the answer key is not legitimate difficulty, because it makes the question unanswerable*.

| Before: as submitted                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | After: as repaired                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>... The modelling code must reconstruct the wake-up forest from the events: when <i>the main thread</i> issues a wake-up a tree is created ... <b>After aggregation</b>, roles whose sample count and critical-hit count reach the threshold are treated as valid dependencies.</p> <p>Each candidate role’s benefit <b>jointly considers</b> the critical wake-up ratio, the critical hit count and the runnable duration. The resource cost is the <b>product</b> of the priority-gap tier count and the activity tier count.</p> | <p>... The <b>main thread is one concrete thread instance, not a batch of same-named threads</b>: take the TID of the thread in the first wake-up event named <code>main_comm</code>, and from then on only that TID is the main thread. After anonymization several threads share that name, while the rest are ordinary threads, and this alone determines the shape of the forest. Node statistics are <b>aggregated into roles by thread name</b>, so that the same name with a different TID still means one role.</p> <p><math>s_r = \frac{c_r}{n_r} \cdot c_r \cdot (1 + \varepsilon q_r)</math>, <math>\varepsilon = 10^{-6}</math>, with <math>q_r</math> in <b>milliseconds</b>. Both tier counts round <b>up</b> and count <b>at least one tier</b>.</p> |

Figure 3: A statement that does not determine its model (HarmonyOS thread scheduling, English gloss of the Chinese original). Left: entity conventions unstated and the tie-breaking constant  $\varepsilon$  present only in the withheld reference formulation. Right: both conventions and the constant made explicit. All four evaluation configurations moved from 0/5 to 5/5.

**Multipliers in the objective.** The datacom load-balancing objective weights each node’s congestion by an importance factor ranging over 0.8–1.2, read from a data field the statement never mentioned. Omitting it is invisible in the formulation, because it is a coefficient rather than a constraint, and coefficients have no ( $C_n$ ) number to fail a consistency check against. Once the statement named the weight and the two-level “peak, then weighted average” structure, the without-annotations configuration went from 0/8 to 8/8. Note what this implies for measurement: before the fix, the ablation of Section 6.5 would have read this statement defect as evidence that the *annotations* carried necessary information.

**Two problems, one statement.** The  $\ell_1$  and  $\ell_2$  video-stabilization problems shipped byte-identical statements. The norm is the only difference between them, it determines the reference solutions, and it appeared in no model input, so every model was guaranteed to be wrong on one of the two. The repair keeps the statements in business language rather than naming a norm: one prices deviation so that “doubling the deviation doubles the cost,” yielding long constant-velocity stretches, while the other penalizes large deviations heavily so the optimizer “spreads unavoidable deviation across every frame.”

**The opposite failure: statements that give the modeling away.** The cross-pod load-balancing statement originally wrote the epigraph linearization into the prose (“traffic may not exceed the product of link capacity and the network-wide maximum utilization”) and handed over the closed-form normalizer  $N(N - 1)/2$ . Both are the modeling work, pre-done. The revision states only the business intent, which is that the operator wants the busiest direction as lightly loaded as possible while building few links, that all links cost the same, and that cost is prorated by the fraction built, and it leaves the formulation to the modeler.

## 4.2 Annotations that mislead

Curated annotations are the one artifact that can be *worse* than absent, because they are read as authoritative. The virtual-power-plant annotations contained a sentence that read as permission to omit the upper-bound half of a positive-part function. With only the lower envelope  $r \geq B_t - P_t$ , the solver pushes the response variable to whatever bound exists and settles unshaved energy as revenue, producing a net cost *below* the true optimum. The rewrite spells out that all three inequalities are required and states plainly that the original observation is “a corollary of writing it strictly, not a reason to omit the upper-bound constraints”. The without-annotations score moved from 13/15 to 15/15. The compiler memory-allocation annotations cited an external address-allocation document, and models followed the citation into address-level variables that tighten the feasible region out of thin air, rendering the entire instance batch infeasible. The rewrite forbids address-layer variables explicitly, and that configuration went from 0/5 to 5/5.

A third failure mode is quieter: the JPEG annotations restated constraints already in the statement. Reviewers flagged that this destroys the ablation’s discriminative power, since both configurations score identically, and the reader cannot tell whether that is because the statement is complete or because the annotations carried no information. The rewrite kept only genuine off-limits knowledge (do not select per image, do not recompute the encoder, do not turn it into a facility-location problem).

## 4.3 Formulations and code that disagree

The compiler memory-allocation submission is the cleanest example of why constraint numbering is checked mechanically. Its code carried three constraints,  $C_{16}$ – $C_{18}$ , linearizing a conjunction of consecutive pipeline choices, while the formulation stopped at  $C_{15}$  and the reference solutions came from the previous generation of the model. Substitution verification failed on all eight instances, the stored solutions referenced nine variables the model no longer contained, and, as the diagnostic detail, the recorded objective was 38% *better* than the model’s true optimum (Figure 4), because the missing  $C_{18}$  let the pipeline-transition variables sit at zero and skip the switching cost they were meant to charge. The lesson generalizes uncomfortably: had the model under test also omitted those constraints and the answer key stayed stale, both sides would have been perfectly self-consistent and every automated check would have passed.

| Before: as submitted                                                                                                                                                                                                                                                                                                                                                                                                   | After: as repaired                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> <b>formulation/model.md</b> ... ### (C14) Local instantaneous capacity ### (C15) Kernel outputs end in Global  <b>code/model.py</b> m.C14 = Constraint(...) m.C15 = Constraint(...) m.C16 = Constraint(...) m.C17 = Constraint(...) m.C18 = Constraint(...)  <b>checks</b> check_model: C16, C17, C18 in code,               absent from model.md verify_solution: 0/8 inst_001 key 63.16 vs model 102.38 </pre> | <pre> <b>formulation/model.md</b> ... ### (C14) Per-scope instantaneous capacity ### (C15) All sinks end in Global ### (C16) <math>z_{a,b,s} \leq h_{a,s-1}</math> ### (C17) <math>z_{a,b,s} \leq h_{b,s}</math> ### (C18) <math>z_{a,b,s} \geq h_{a,s-1} + h_{b,s} - 1</math>  <b>code/model.py</b> m.C14 ...m.C18 = Constraint(...)  <b>checks</b> check_model: pass  verify_solution: 8/8 inst_001 key 102.38 = model 102.38 </pre> |

Figure 4: Formulation and code disagreeing (compiler memory allocation). The code carried three constraints linearizing a conjunction of consecutive pipeline choices, while the formulation stopped at  $C_{15}$  and the answer key came from the previous model generation. The missing  $C_{18}$  is the forcing direction, so the stored objective was 38% *better* than the model’s true optimum. Had the model under test omitted the same constraints, both sides would have been self-consistent and every check would have passed.

Code defects are less exotic but equally consequential. One submission’s `build()` placed a maximum expression directly in the objective instead of introducing an epigraph variable, so the classifier read the model as nonlinear, routed it to MINLP solvers, and both failed to return anything, while the submitter’s private solve path linearized separately, meaning the reference solutions came from a different model than the one the toolchain consumes. Another silently defaulted objective weights to hard-coded values when an optional data file was absent, so an instance shipping without that file would be scored against weights no modeler could see.

#### 4.4 Answer keys that cannot be trusted

This is the most dangerous class, because a wrong key fails correct models silently and permanently.

**A heuristic labeled as an answer.** An early sample problem shipped a greedily constructed schedule of cost 906 as its reference, whereas the true optimum was 302. Any model finding 302, which is the right answer, would have been marked wrong by a factor of three. The honest *heuristic* status label saved it (such instances are excluded from judging rather than trusted), which is exactly why the three-value status discipline exists, and the problem was ultimately removed rather than repaired.

**Optimality that the solver never proved.** Four microgrid instances carried status: `optimal` with gap: `0.0` (Figure 5). They were not optimal: the solver’s default  $10^{-4}$  relative MIP gap, on objectives of magnitude  $9.3 \times 10^6$ , admitted absolute errors of 622–735, a relative error of  $6.6\text{--}7.8 \times 10^{-5}$ , which is 66–78 times the  $10^{-6}$  scoring tolerance. A correct model would have been failed. Three details make this case worth reporting. It was not caught by a tool but by a contradiction: one instance’s data was a strict superset of another’s, yet reported the *smaller* optimum. Substitution verification could not have caught it, because the

solution is genuinely feasible and its objective is genuinely its own. And the optimality audit was fooled too, because it re-solved at the same default tolerance and certified agreement. It now pins the gap to zero, and under that setting it correctly reports four of ten as contradicted. Reference solves now pin the gap to zero at generation time as well.

| Before: as submitted                                                                                                                                    | After: as repaired                                                                                                                                 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>solution/inst_003.json</b><br><pre>{ "status": "optimal",   "objective": 9297616.756461179,   "bound": 9297616.756461179,   "gap": 0.0 }</pre>       | <b>solution/inst_003.json</b><br><pre>{ "status": "optimal",   "objective": 9296946.424402560,   "bound": 9296946.424402560,   "gap": 0.0 }</pre>  |
| <b>code/model.py</b><br><pre>result = opt.solve(m, ...) if tc == "optimal":     status, bound, gap =         "optimal", objective, 0.0</pre>            | <b>code/model.py</b><br><pre>opt.highs_options = {     "mip_rel_gap": 0.0,     "mip_abs_gap": 0.0 }</pre>                                          |
| <b>consequence</b><br><pre>solver default mip_rel_gap = 1e-4 true optimum 9296946.42 error 670 = 7.2e-5 relative = 72x the 1e-6 scoring tolerance</pre> | <b>consequence</b><br><pre>gap pinned to zero at generation and in the optimality audit audit now reports 4 of 10 contradicted, instead of 0</pre> |

Figure 5: An answer key that claims optimality the solver never proved (microgrid sizing). A correct model finding 9296946.42 would have been failed against the recorded 9297616.76. Substitution verification cannot catch this, since the solution is genuinely feasible and its objective genuinely its own, and the optimality audit was fooled too, until it also pinned the gap to zero. It surfaced from a contradiction: one instance’s data was a strict superset of another’s, yet reported the smaller optimum.

**Constructed schedules standing in for solves.** The four largest microgrid instances recorded hand-constructed feasible schedules (solver: "constructed feasible schedule", time\_limit\_sec: 0). On one of them a genuine 600-second solve found an objective of  $6.55 \times 10^6$  against the recorded  $86.7 \times 10^6$ , so the key was 13× the optimum, and any competent model would appear to “beat” it by an order of magnitude and the comparison would measure nothing. Re-solving with the recorded solution as a warm start repaired three of the four, while the fourth exhausts memory at  $5.3 \times 10^6$  variables-plus-constraints and *still carries the constructed schedule today*, which we report as an open defect rather than quietly excluding.

**A key beaten by the prompt’s own hint.** One cross-pod instance’s reference was 23.7% worse than a construction that the data dictionary, an input the model under test receives, describes outright. A model that simply followed the hint would have produced a better solution than the answer key and been judged wrong for it. The repair re-solved the instance to match the construction, and a later systematic re-solve improved seven of the ten references further and, on one instance, replaced a vacuous bound: 0.0, gap: 1.0 with a real bound.

**Fabricated bounds.** One submission’s solution writer filled bound with the objective and gap with 0.0 whenever no dual bound was available, which turned every feasible solution into a claim of proven optimality. This is a mechanism defect rather than a data defect: uncorrected, it would have mislabeled every future reference the dataset produced.

## 4.5 Instances that cannot discriminate

A problem can be flawless and still measure nothing, if its instances never exercise its constraints. The JPEG quantization-table problem was first posed as choosing one table from a pre-enumerated candidate list. That formulation degenerates: with a single “select exactly one” constraint, the optimum is an argmin, and a five-line scan over the candidates reproduced all eight reference objectives exactly. Two of its four constraints made it worse, in that the PSNR and FSIM floors were satisfied by every candidate on every instance, so deleting them changed nothing. An earlier revision of the same problem had also shipped the answer inside a model input, with a data field whose value matched the reference objective digit for digit. Reposing the problem as 64 independent per-frequency step-size choices, which form a multi-choice knapsack whose search space is the product over frequencies and is coupled across images by the quality floor, made the quality constraint dominant: a greedy solution now violates it by a factor of 18–28 (Figure 6).

| Before: as submitted                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | After: as repaired                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>formulation/model.md</b><br>decision: $x_t \in \{0,1\}$ over candidate tables $T$<br>objective: $\min \sum_{i \in I} \sum_{t \in T} S_{t,i} x_t$<br>(C1) $\sum_{t \in T} x_t = 1$ exactly one table<br>(C2) PSNR floor per image<br>(C3) FSIM floor per image<br>(C4) $Q_k = \sum_t q_{t,k} x_t$<br><b>what it measures</b><br>argmin over $ T $ candidates<br>5-line scan reproduces 8/8 references<br>C2, C3 satisfied by every candidate<br>$\Rightarrow$ deleting them changes nothing | <b>formulation/model.md</b><br>decision: $x_{k,c} \in \{0,1\}$ , step $c$ at frequency $k$<br>objective: $\min \sum_i \sum_k \sum_c R_{i,k,c} x_{k,c}$<br>(C1) $\sum_{c \in C_k} x_{k,c} = 1 \forall k$ per frequency<br>(C2) MSE ceiling per image<br>(C3) $Q_k = \sum_c s_{k,c} x_{k,c}$<br><b>what it measures</b><br>multi-choice knapsack, $\prod_k  C_k $<br>greedy violates C2 by 18–28×<br>images coupled through C2<br>$\Rightarrow$ the quality floor now decides |

Figure 6: A problem whose structure could not discriminate (JPEG quantization-table design). Left: as first submitted, selecting one table from a candidate list, which is an argmin that a five-line scan solves, with two of four constraints never binding on any instance. Right: reposed as 64 per-frequency step choices coupled by a shared quality ceiling.

Instance data degenerates in the same way. One load-balancing instance shipped two path configurations per service over *identical* link sets, so all four decision combinations produced identical link utilizations and a congestion term that was constant at 0.503125: the entire model body, covering link utilization, per-node time-slot peaks and hard capacities, was inert, and what looked like a routing problem was a binary judgment (Figure 7). After the paths were regenerated over distinct link sets, all three constraint families reached 8/8 coverage.

| Before: as submitted                                                                                                                                                                                                                                                                                                                                                                                                            | After: as repaired                                                                                                                                                                                                                                                                                                                                                |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>data/inst_001/path.csv</b><br>id,src,dst,type,weight,...,links<br>1,1,5,1,0.4422,...,1;3;4<br>2,1,5,2,0.6199,...,1;3;4<br>3,2,5,1,0.5250,...,2;3;4<br>4,2,5,2,0.8940,...,2;3;4<br><br><b>consequence</b><br>both options of a service traverse<br>the same links $\Rightarrow$ all four decision<br>combinations give identical utilization,<br>with congestion constant at 0.503125<br>$\Rightarrow$ model body non-binding | <b>data/inst_001/path.csv</b><br>id,src,dst,type,weight,...,links<br>1,1,6,1,0.6199,...,1;5;7<br>2,1,6,2,0.6423,...,2;6;7<br>3,2,6,1,0.8940,...,3;6;7<br>4,2,6,1,0.8081,...,3;6;7<br>5,2,6,2,0.8014,...,4;5;7<br><br><b>consequence</b><br>options differ in the links they use<br>$\Rightarrow$ the choice moves utilization<br>drop-one coverage C1/C2/C3 = 8/8 |

Figure 7: Instance data that cannot discriminate (datacom load balancing). Left: as submitted, the two configurations offered for each service ran over the same link sets. Right: regenerated over distinct link sets, so the decision the benchmark asks for actually changes the objective.

A submission still under review illustrates a subtler variant: its objective magnitudes (0.27–5) sat in the same range as the audit’s cut tolerance ( $10^{-6}$ ) and the solver’s feasibility tolerance, producing spurious audit failures, while its residual-threshold constraint never bound on any of 160 pilots across eight instances. The fix was not to loosen tolerances but to redesign the objective into an integer count, after which the audit passed cleanly.

#### 4.6 What the repairs bought, and what they cost

Table 2 is the accounting: 40 distinct defects across 13 classes, over 101 pull requests of which 92 merged. The distribution is the argument for the toolchain. 23 of 40 were caught by a script that runs on every pull request, which is the case for building the scripts at all: at this volume, a defect class that depends on a reviewer noticing will eventually ship. But the split is not clean in the direction one would hope. Coverage auditing catches a class no consistency check can see, and it is the second most productive gate. And the statement-fidelity class, the largest single class at 10, is caught by no gate: a statement that under-determines its model leaves formulation, code, and solution perfectly self-consistent, so every script passes. It took an independent re-derivation pass, and for several problems the closed-book evaluation results themselves, to find them. A corpus is only as good as the class of errors someone went looking for, and the mechanical gates define the classes we stopped having to look for by hand.

The accounting should be stated honestly in both directions. Across the ten revised statements, the with-annotations configuration improved from 55/72 to 71/72. The without-annotations aggregate did *not* improve: it stayed at 55/72, with large gains on three problems offset by regressions on three others, at least one of which is a code-generation failure rather than a modeling error, and all of it measured in single runs without seed control. We report the flat aggregate rather than the favorable half. Two further caveats belong here. Completing a statement legitimately *reduces* the discriminative power of the annotation ablation, since the without-annotations configuration now receives information it previously lacked. We accept that trade because unanswerable is not the same as difficult. And seven of 17 statements needed no change at all: an audit that finds nothing in seven cases and something in ten is a calibrated audit, not a failed one.

Table 2: Every defect class we encountered, with its occurrence count and the gate that caught it. Counts are distinct defects, not affected instances: one stale answer key failing eight instances is one defect. The first eight rows are caught by a script, while the rest need an adversarial reading, an ablation, or the evaluation itself.

| Defect                                           | Caught by                                    | #  | Consequence if shipped                                                                        |
|--------------------------------------------------|----------------------------------------------|----|-----------------------------------------------------------------------------------------------|
| Formulation–code mismatch                        | Constraint-numbering check (mechanical)      | 1  | Wrong model and stale key agree, while every other check passes                               |
| Solution infeasible or mis-valued                | Substitution verification (mechanical)       | 2  | Key fails correct models                                                                      |
| Fabricated bound or gap                          | Substitution verification (mechanical)       | 1  | Every feasible solution mislabelled as proven optimal, for all future references              |
| Unproven optimality claim                        | Zero-gap re-solve audit                      | 4  | Correct models scored wrong by up to 78x the scoring tolerance                                |
| Heuristic standing in for a solve                | Status discipline plus re-solve audit        | 5  | Comparison measures nothing, and a competent model appears to beat the key                    |
| Answer key improvable from the prompt’s own hint | Reference replay under the evaluation regime | 1  | A model following the hint is judged wrong for beating the key                                |
| Non-binding constraint family                    | Drop-one coverage audit                      | 7  | A model omitting the family scores perfectly                                                  |
| Degenerate problem or instance structure         | Drop-one coverage audit                      | 2  | The benchmark measures something far easier than it appears to                                |
| Statement under-determines the model             | Independent re-derivation (human)            | 10 | Task unanswerable, and the annotation ablation misattributes the cause                        |
| Statement leaks the formulation                  | Human review                                 | 1  | Difficulty removed, so the benchmark inflates scores                                          |
| Misleading annotations                           | With/without ablation                        | 3  | Models steered into infeasible or inflated models, or the ablation loses discriminative power |
| Code defect in the reference implementation      | Consistency check plus human review          | 2  | Reference solutions come from a different model than the toolchain consumes                   |
| Judge error                                      | Judge audit plus offline re-judgment         | 1  | Correct models failed, which is indistinguishable from a modeling error without a judge audit |

## 5 Verification and Evaluation Methodology

Section 4 showed the defects a benchmark of this kind accumulates. This section is the other half of that story: the gates built to stop them, why each one exists, the design idea that makes it cheap enough to run on every pull request, and what it actually caught. It then states what the evaluation protocol gives a model under test and what a verdict certifies.

We describe the methodology rather than ship it. The corpus is released, comprising statements, data, reference formulations, reference implementations, reference solutions, provenance cards and coverage records, but the verification and evaluation toolchain is not (Section 5.5). That makes the reasoning below the deliverable, so we give each gate the four things a reader would need to rebuild or adapt it: the failure it exists to catch, the idea that makes it work, the boundary of what it proves, and its yield on our own corpus.

## 5.1 Four gates on the construction chain

From business need to certified answer, every problem crosses three translations, and each has its own gate, because each fails differently (Figure 8). A fourth gate checks not the artifacts but the instances: whether they can detect a defect at all.

**(1) Prose to mathematics: no gate exists, and that is the finding.** A business rule that never reaches the formulation leaves formulation, code and solution perfectly consistent with each other, so no check over the artifacts can see it. This is the largest defect class we encountered (10 of 40, see Table 2) and the only one with no mechanical gate. What we have instead is procedural: an LLM reviewer works a written checklist on every pull request and posts findings until none block, then a human maintainer reviews and merges, since the machine never approves and only asks the routine questions first, plus a periodic re-derivation pass (Section 4.1). Anyone building a corpus like this should budget for the fact that their most common defect will be the one their tooling cannot detect.

**(2) Mathematics to code: make the correspondence nominal.** *The failure:* a constraint present in the formulation but not the code, or the reverse, which yields a model that disagrees with its own documentation and an answer key generated from whichever side the submitter happened to run (Section 4.3). *The idea:* checking semantic equivalence between  $\text{\LaTeX}$  and Pyomo is undecidable in practice, so we do not try. Requiring every constraint family to be numbered ( $C_n$ ) in the formulation and named `Cn` in the code turns the correspondence into a *nominal* one, and a set comparison then catches the entire class in milliseconds with no solver. The convention costs submitters almost nothing and converts an intractable check into a trivial one. A deliberate omission must be declared with an explicit `exempt` marker naming the constraint, so “not implemented” becomes a statement someone signed rather than an absence. *Boundary:* it proves the two artifacts mention the same families. It proves nothing about whether `C7` in the code says what ( $C_7$ ) in the formulation says. *Yield:* the compiler memory-allocation submission, whose code carried three constraints the formulation lacked, with an answer key 38% better than the model’s true optimum.

**(3) Code to solution: substitute, do not solve.** *The failure:* a reference solution that is infeasible, or whose recorded objective is not the objective its own variable assignment produces. *The idea:* verifying a solution does not require solving. Rebuild the model, substitute the recorded assignment, re-evaluate every constraint and the objective to  $10^{-6}$ . This separates the cheap deterministic question (is this solution feasible and honestly priced?) from the expensive one (is it optimal?), and the cheap half needs no solver at all, which is why it runs in CI on every pull request, on every instance, including instances of  $5 \times 10^6$  variables that no CI runner could solve. *Boundary:* feasibility and honest bookkeeping only. A feasible, honestly priced, thoroughly sub-optimal solution passes.

**(4) Optimality: turn a claim into a feasibility question.** *The failure:* a key labelled `optimal` that is not, which fails correct models silently and permanently, and is the most dangerous class, because the corpus is self-consistent and only a better solver disagrees. *The idea:* cut and prove. To test a claimed optimum  $z^*$ , add the constraint that the objective must strictly beat  $z^*$  and re-solve, and if that model is infeasible, the claim holds. An optimality claim becomes a feasibility question, which solvers answer faster and more reliably than they re-derive the optimum. Two disciplines make it sound. Solutions honestly labelled `heuristic` are exempt and excluded from judging instead (Section 5.3), so honesty is never punished. And the gap is pinned to zero, a lesson learned the hard way, as described below. *Boundary:* within a time budget, so its verdicts are *contradicted*, *consistent*, or *unproven*, and the third is not evidence of correctness. *Yield, and a lesson about the auditor auditing itself.* The audit’s history is worth stating precisely, because it is easy to quote a reassuring version of it. Run at the solver’s default MIP gap over the 13 problems admitted

at the time, it contradicted nothing. That was not evidence of a clean corpus but of a tolerance set too loose to see: an auditor inheriting the default  $10^{-4}$  gap certifies exactly the errors that gap admits, and on objectives of magnitude  $10^7$  that is four orders of magnitude coarser than the scoring tolerance it is supposed to protect. Once the gap was pinned to zero, the same audit contradicted four of ten microgrid instances (Section 4.4), which were then re-solved and replaced. The audit as it stands today, with the gap pinned to zero and re-run over the whole release for this paper, reports 109 consistent, 15 unproven and 0 contradicted (Appendix B), with 4 heuristic keys exempt and one disclosed exception carried as an open defect rather than resolved (the largest microgrid instance, Section 4.4). We report the unproven column separately rather than rounding it into the zero, since that is exactly the error the default-gap run made.

This discipline is not hypothetical: an early sample problem shipped a heuristic schedule of cost 906 labeled as a reference answer whose true optimum was later proven to be 302, and an evaluator trusting that label would have failed every correct model. The audit exists so that this class of error cannot enter the benchmark.

## 5.2 Closed-book evaluation

The end-to-end task: given only the problem statement, the data dictionary, and a single sample instance for format (optionally plus annotations.md), the model under test must produce a `Pyomo build(data)` function. We then run that code ourselves on every instance, because model-reported numbers are never trusted.

Evaluation runs in an isolated cloud sandbox with no repository access, and each run’s execution environment is verified to be that sandbox before its result counts. That verification exists because we were burned without it: a silent SDK fallback once ran the agent locally with the full repository mounted, leaving reference formulation, code, and solutions all readable, and the “generated” model came back verbatim identical to the reference implementation, with no error raised anywhere. The lesson generalizes past our setup: an evaluation that cannot prove its own isolation is not closed-book, and the failure is silent. All numbers in this paper come from runs after that hole was closed. Every run archives the prompt actually sent, the raw response, the extracted code, per-instance results, and solver logs, so any reported number can be traced to the artifacts that produced it.

## 5.3 The judging chain

A generated model passes on an instance only after surviving four gates:

1. **Solvable.** The generated `build(data)` constructs and solves in a sandboxed worker under the pinned solve regime below.
2. **Feasible in its own terms.** The solution is substituted back into the *generated* model and every constraint is re-checked, since a solver status line is not taken at its word.
3. **Cross-validated against the reference.** The solution is substituted into the *reference* model, catching models that omit a constraint yet land on a coincidentally matching objective. Formally, let  $\hat{x}$  be the generated solution and  $V, \hat{V}$  the variable families of the reference and generated models. The check is defined only when the two name spaces correspond, meaning  $V = \hat{V}$  up to families that are entirely absent on one side, and returns

$$\text{xval}(\hat{x}) = \begin{cases} \text{confirmed} & \text{all reference constraints hold under } \hat{x}, \\ \text{violated} & \text{some reference constraint is violated,} \\ \text{abstain} & \text{a family is present on one side only.} \end{cases}$$

Abstention is a verdict, not a failure: substituting away an auxiliary variable is legitimate modeling, so a mismatch in variable spaces is evidence of a different modeling rather than of a wrong one, and

guessing in either direction would be worse than declining. Only *violated* fails an instance. The direction symmetry is a lesson learned, not a flourish: the first deployment abstained only when the generated solution introduced unknown variables, and zero-filled reference variables missing from the generated solution. Substituting away an auxiliary variable is legitimate modeling, so this misjudged correct models, and 36 instance-runs in one archived sweep raised false infeasibility alarms while 30 verdicts flipped when the archived runs were re-judged off-line, without re-running a single model. We report this because a judge that has never been audited should not be assumed correct, in this benchmark or any other.

4. **Objective match.** The objective agrees with the reference optimum within  $10^{-6}$  relative or absolute tolerance. Decision variables are deliberately *not* compared: combinatorial problems routinely have multiple optima, and comparing  $x$  would fail correct models.

Only instances whose reference solution is proven optimal are judgeable, while *feasible* and *heuristic* references make the instance *unjudgeable* rather than silently weakening the answer key (15 of 128 instances in the current release).

**Solve regime.** Every generated and reference model is solved under one pinned regime, reported here because pass rates are meaningless without it: solvers are routed by model class, sending LP/MILP to HiGHS then SCIP, QP/NLP to Ipopt then Couenne, and MINLP to SCIP then Couenne, with 900 s of wall clock per solver and the MIP gap tightened to  $10^{-6}$  to match the scoring tolerance, since a solver stopping at its default  $10^{-4}$  gap would misjudge a correct model. The regime and the dataset content are both version-pinned, and a stored result is reused only while both are unchanged, so revising the judge re-runs exactly the population it affects rather than silently mixing verdicts from two different judges.

## 5.4 Constraint coverage: does the benchmark itself discriminate?

A benchmark instance only tests a constraint if omitting that constraint changes the answer. We make this measurable: each constraint family is disabled in turn, the instance re-solved, and the fraction of instances on which the optimum moves is recorded. A family with zero coverage is a blind spot, because a model that never writes it scores perfectly.

Formally, for a problem with instance set  $I$  and constraint families  $\{C_n\}$ , let  $z(i)$  be instance  $i$ 's optimum and  $z_{-n}(i)$  its optimum with family  $C_n$  deactivated. The coverage of  $C_n$  is

$$\text{cov}(C_n) = \frac{1}{|I|} \left| \{i \in I : z_{-n}(i) \neq z(i) \text{ or } i \text{ becomes infeasible or unbounded}\} \right|,$$

evaluated at a tolerance of  $10^{-9}$  on the objective. A family with  $\text{cov}(C_n) = 0$  is undetectable: a submission omitting it scores exactly as a correct one does, on every instance the problem has. The metric needs nothing from IndusOPT beyond a reference model and instance data, so any benchmark that ships executable references can compute it.

Zero coverage has two causes, and they call for opposite responses. The instances may be too weak, in which case the repair is instance design, which is what happened on the datacom load-balancing problem, whose duplicated path sets left three families inert until the paths were regenerated (Section 4.5). Or the family may be *implied by the rest of the model*, in which case no instance can ever make it bind and there is nothing to repair. Our one zero-coverage family is of the second kind, and we could only tell the two apart by asking why. In the HarmonyOS thread problem,  $C_2$  forbids boosting a role whose dependency indicator  $d_r$  is zero, while the objective is  $\max \sum_r d_r s_r y_r$ : such a role contributes exactly zero and still consumes budget in  $C_1$ , so relaxing  $C_2$  frees only zero-valued variables at nonnegative cost and the optimum cannot move. Across its five instances, 2,644 of 3,080 roles have  $d_r = 0$  and the largest objective coefficient among them is 0. We keep  $C_2$ , because it states a rule the business statement does assert, and publish it as a permanent blind spot: a

submission omitting it scores perfectly here, and always will. The distinction matters for anyone adopting the metric, since a coverage report that does not separate weak instances from redundant constraints will send people to redesign data that cannot be fixed.

Coverage auditing has caught real blind spots repeatedly, and it quantifies partial ones (e.g. a link-protection exclusivity constraint binding on only the smallest of five instances, and a pivot-block condition binding on one of six). We report per-problem coverage in Section 6, the complete per-family record in Table 6, and treat weak coverage as a published limitation rather than a private embarrassment: pass rates on a zero-coverage constraint are exactly as informative as the coverage number says they are. The audit is not complete, since it covers 12 of 17 problems, and Section 7 explains why the metric runs out of budget precisely on the largest ones.

To our knowledge no existing NL-to-optimization benchmark measures its own discriminative power this way, and the neighbouring checks answer different questions: MIPLIB-NL validates that its generated text is *sufficient* to reconstruct the model [22], and Opti-Agent-Bench checks a submission for consistency across its own modules [12]. Both ask whether an artifact is self-consistent, whereas coverage asks whether the benchmark can tell if it is wrong. The three are complementary, and a corpus that ships executable reference models can run all of them.

## 5.5 What is released, and what is not

IndusOPT is released for a specific purpose: so that people building optimization-modeling systems have an industrial-scale corpus to measure themselves against. It is not released as a construction kit. The two halves of the project are therefore treated differently.

**Released: the whole corpus, keys included.** Statements, annotation files, data dictionaries, and all 128 instances go out, together with the registration card of each problem with its provenance and desensitization record. And, in the decision that matters most, so do the reference formulation, the reference Pyomo implementation, and the reference solutions for every instance. Alongside them ship the per-problem measurements this paper reports: model class, sizes, instance axes, and the drop-one coverage record as a `coverage.json` in each problem directory, blind spots included.

Shipping the keys is forced by the decision not to ship the toolchain. A corpus with neither answers nor an evaluator is not a benchmark anyone can use, and withholding both would mean every prospective user waits on us. We accept the cost this carries, namely that public keys are contaminable and that a corpus on the public internet will eventually be in a pretraining set, and we address it in Section 5.6 with versioning rather than secrecy.

**Not released: the toolchain.** We give the design and reasoning above, and report what each gate found including where it was wrong, but do not ship the implementation. The two halves have genuinely different reasons and we separate them rather than offering one that only covers half.

For the *closed-book evaluation harness and the four-gate judge*, the reason is the ordinary one: a benchmark whose judge is public is a benchmark that can be optimized against rather than solved. Scoring logic, prompt templates, and the isolation checks are the parts worth gaming, and they stay with us.

For the *construction-side quality gates*, meaning consistency, substitution, optimality audit and coverage, that reason does not apply. They run when a problem is admitted, not when a submission is scored, so there is no adversary and nothing to game. The honest reason is narrower: they are coupled to this repository’s review records and CI, several of which reference internal systems, and we are not prepared to maintain them as a general-purpose package. That is a resourcing decision, not a security one, and it costs little. Each is a modest amount of code over artifacts we publish: given the released reference models and solutions, the substitution check is a straightforward traversal of a Pyomo model, and the cut-and-prove audit and the drop-one loop are short scripts over a solver. We have specified all four precisely enough above to be re-implemented, and would rather someone re-implement them against their own corpus than adapt ours. What we contribute here

is the design rationale and the empirical finding that each gate earns its place (Table 2), rather than the source. One consequence we accept: our reported numbers cannot be reproduced by re-running our judge. What can be checked is everything the judge consumed and produced. The corpus is public, the archived per-run records are public, and the headline numbers in this paper are recomputed from those archives by `paper/gen_numbers.py` rather than transcribed, so a reader who disputes a number can recompute it from the artifacts. The claim we make is falsifiability, not reproducibility of the pipeline.

## 5.6 Availability, licensing, versioning, and contamination

**Availability.** IndusOPT v1.0 is the tagged release identified in Section 3.6 (4d851173f8-dirty), distributed from the project’s public repository and archived under a DOI so that a citation resolves to a fixed corpus rather than to a moving branch. The repository is the working copy and keeps changing, while the archived release is what this paper measures.

**Licensing.** Problem statements, annotations, data dictionaries, instance data, formulations and reference solutions are released for research use under a permissive content licence, with the reference implementations under a permissive software licence. Third-party inputs keep their own terms and are not relicensed by us: the SuiteSparse matrices in the two pivoting problems and the Kodak images behind the JPEG rate–distortion tables are redistributed under their original conditions, and the registration card of each problem records its upstream source. Two problems carry internal sensitivity (Section 3.4), and their inclusion in the public release follows the same desensitization review as the rest and is recorded per problem rather than assumed.

**Versioning.** Releases are tagged and immutable. Problems are added between releases, and a new release is cut only together with a re-run of the evaluation sweep, so that the corpus and the reported numbers never describe different populations. Reference solutions are the one artifact that improves within a release line: when a replay under the pinned regime finds a strictly better incumbent, warm-started from the recorded solution so it can only match or improve (Appendix F), the key is updated under the audit trail. Corpora that freeze their keys at release accumulate exactly the error class of Section 4.4.

**Contamination.** Public keys are contaminable, and we prefer saying so to pretending otherwise. Until now the data has never been public, so the numbers in this paper are uncontaminated by construction, but that property expires the moment the corpus is posted, and it expires for every benchmark that publishes its answers. Three measures rather than one. The release carries a canary string, so that anyone auditing a future model’s training data can test for inclusion directly. Every result reports the release version it was measured on, so a score on v1.0 obtained after v1.0 entered the pretraining corpora is not silently comparable with ours. And subsequent releases will hold out a private instance split per problem, that is, instances rather than problems, so the held-out set exercises the same modeling with data no submission has seen. We regard none of these as a solution. The honest position is that a public benchmark measures uncontaminated capability once, and that the value it has afterwards comes from versioning discipline rather than from secrecy.

**Maintenance.** The repository accepts new problems through the review pipeline of Section 5.1, and the same gates apply to submissions from outside the original team. Defects found after release are recorded against the problem rather than silently patched, and the open microgrid defect of Section 4.4 is carried this way.

## 6 Experiments

### 6.1 Setup

We report the most recent full closed-book sweep: two commercial coding models, namely `claude-opus-5` and `composer-2.5`, both accessed through the same cloud-agent harness, each evaluated with and without `annotations.md`, over all 17 problems (68 runs, 512 instance-runs). Of the 128 instances, 113 are judgeable per configuration under the answer-key rule of Section 5.3. All numbers below are under the repaired

Table 3: Passed / judgeable instances per problem and configuration (“w/” = with annotations, “w/o” = without). Post-re-judgment verdicts, recomputed from the archived runs. The primary condition is *without* annotations (Section 6.3). Three summary rows report what a pass is worth. The first is the judgeable denominator, which is the number comparable to other benchmarks. The second counts all 128 instances with the 15 unjudgeable ones counted as not passed, which is the conservative bound. The third is the structurally verified rate, counting only passes whose solution the bidirectional cross-check actively confirmed inside the reference model.

| Problem                        | Class | Judg. | composer-2.5 |           | claude-opus-5 |           |
|--------------------------------|-------|-------|--------------|-----------|---------------|-----------|
|                                |       |       | w/           | w/o       | w/            | w/o       |
| SparseLA-LDLSymmetricPivoting  | IP    | 6/6   | 6            | 6         | 6             | 6         |
| SparseLA-LUPivotReordering     | IP    | 6/6   | 6            | 6         | 6             | 6         |
| Camera-JPEGQuantizationTable   | MILP  | 8/8   | 0            | 8         | 8             | 8         |
| Camera-VideoStabilization-L1   | LP    | 8/8   | 8            | 6         | 8             | 8         |
| Camera-VideoStabilization-L2   | QP    | 8/8   | 8            | 8         | 8             | 8         |
| Communication-RailCellHandover | MILP  | 5/5   | 5            | 5         | 5             | 5         |
| HarmonyOS-CriticalThreadOpt    | MILP  | 5/5   | 0            | 0         | 0             | 0         |
| HarmonyOS-MemoryEviction       | MILP  | 8/8   | 8            | 8         | 8             | 8         |
| Cluster-CrossPodLoadBalancing  | MILP  | 3/10  | 3            | 3         | 3             | 0         |
| TBE-MemoryAllocation           | MILP  | 5/8   | 5            | 0         | 0             | 4         |
| Microgrid-SizingAndOperation   | MILP  | 6/10  | 6            | 6         | 0             | 6         |
| VPP-DayAheadLoadScheduling     | MILP  | 15/15 | 0            | 12        | 15            | 13        |
| DataCom-LoadBalancing          | MILP  | 8/8   | 8            | 0         | 8             | 0         |
| DataCom-CapacityExpansion      | MILP  | 8/8   | 8            | 8         | 8             | 8         |
| LLM-MoEExpertLoadBalance       | MINLP | 4/5   | 3            | 2         | 3             | 3         |
| OpticalNetwork-LinkProtection  | MILP  | 5/5   | 5            | 2         | 5             | 5         |
| OpticalNetwork-PathProtection  | MILP  | 5/5   | 5            | 5         | 5             | 5         |
| <b>Total passed</b>            |       |       | <b>84</b>    | <b>85</b> | <b>96</b>     | <b>93</b> |
| Pass rate (of 113 judgeable)   |       |       | 74.3%        | 75.2%     | 85.0%         | 82.3%     |
| Pass rate (of all 128)         |       |       | 65.6%        | 66.4%     | 75.0%         | 72.7%     |
| <i>Structurally verified</i>   |       |       | 13.3%        | 5.3%      | 12.4%         | 5.3%      |

bidirectional cross-check. The archived per-run records retain the original verdicts, and the re-judgment is itself an archived, reproducible artifact. The harness exposes neither temperature nor seed, so each cell is a single observation rather than a distribution, and we return to this in Section 7.

## 6.2 Main results

Table 3 gives the per-problem matrix, and Figure 9 shows its structure at a glance. The stronger model passes 96/113 judgeable instance-runs (85.0%) with annotations and 93/113 (82.3%) without, while `composer-2.5` passes 84/113 and 85/113. We take the headline seriously in both directions: a frontier coding model inside a capable agent harness now solves most of a forward-built industrial corpus, and that is a real change from the 26.9% bare-LLM average (and the at-most-5.5% specialist scores) reported on reverse-constructed industrial instances [22]. It is also not the same statement as *the task is solved*, and the benchmark is instrumented well enough to say precisely how the two differ.

**The headline is measured on the easy subset, and we report the conservative bound alongside it.** The 113 judgeable instances exclude the 15 whose reference solutions the strongest available open-source solvers could not prove optimal, and those are not distributed at random. All 15 come from four problems (cross-pod balancing, TBE memory allocation, microgrid sizing, MoE placement), which are among the largest in the

benchmark. The judgeable set is therefore biased toward what is easy *to verify*, which correlates with what is easy to model. Counting every instance and scoring the unverifiable ones as not passed, the same runs give 75.0% and 65.6%. Both denominators appear in Table 3, where the first is the comparable number and the second is the honest ceiling. The commercial-solver replay of Section 6.9 would move instances from the second into the first, and we expect it to lower the rate rather than raise it.

**Part of the score is bought with information a deployment does not have.** `annotations.md` is written by the engineer who owns the system, and it contains exactly what a requirement author forgets to say. In the field that document does not exist at the moment the requirement is handed over, and it is recovered, if at all, through iteration. The no-annotation column is thus the closer analogue of first contact with a real requirement, and it is the primary condition here. Aggregate movement understates the effect, because it nets out problems that move in opposite directions: the ablation is worth whole problems where it bites, and on the virtual-power-plant problem it is worth a whole problem in reverse (Section 6.5). The single sharpest instance we originally reported, datacom load balancing at 8/8 with annotations and 0/8 without, turned out on repeated runs to be partly a defect in the statement rather than a property of the annotations, and Section 6.8 withdraws it.

### 6.3 What a pass is worth: two tiers, and which one is the headline

A pass certifies less than it appears to, and because IndusOPT stores full reference solutions we can say exactly how much less. Of the 512 instance-runs, the bidirectional cross-check *actively confirmed* the generated solution inside the reference model on 42. It abstained on 401 because the two variable spaces do not correspond, and never ran on the remaining 69, whose references are not proven optimal. It found 0 true violations. So the great majority of passes in Table 3 attest objective agreement plus feasibility of the model’s own solution, which is compatible with a structurally wrong model that lands on the right optimum, and only a small minority attest agreement with the reference *model*.

We therefore report two tiers and are explicit about which claim each supports. The *consistent* tier is the pass rate of Table 3: the generated model solves, its solution is feasible in it, and the objective matches the reference to  $10^{-6}$ . This is the number comparable to the rest of the literature, and it is a permissive upper bound. The *structurally verified* tier additionally requires the cross-check to have confirmed rather than abstained, and it is far lower: 5.3% for `claude-opus-5` without annotations, against 82.3% consistent. That gap must not be read as models failing 95% of the time. It is overwhelmingly the judge’s reach, not the model’s error: the cross-check can only run when the generated model names variables the reference also has, so it reaches problems whose indexing follows the data entities directly and abstains on legitimate reformulations. The structurally verified passes come from just 3 of 17 problems. What the strict tier is good for is bounding the claim: on 3 problems we can say a pass means the model agrees with ours, and everywhere else we cannot, and no amount of additional passing changes that. It also does not saturate, which the consistent tier is close to doing.

Two consequences for how we report. First, the primary condition throughout is *without* annotations, because that is the one that corresponds to a real requirement handed over for the first time (Section 6.5), and the with-annotations column is the diagnostic, not the score. Second, neither tier escapes the partial-oracle problem, and the coverage audit of Section 6.7 shows why: one constraint family is provably undetectable, so a model omitting it scores perfectly under both tiers. The distinguishing property of IndusOPT is not that this problem is absent, since it is present in every benchmark in this literature, but that the benchmark measures its own reach and publishes where the verdict is weak. Benchmarks that ship no reference models (Section 2) cannot run either audit at all.

**The residual is concentrated, and the resolution is coarse.** Most problems are solved perfectly by both models in all configurations, while a small set absorbs almost all failures, namely thread scheduling from raw kernel traces, the MINLP expert-placement problem, and the two problems with heuristic reference answers.

The all-zero problem (`CriticalThreadOpt`) fails not on arithmetic but on world-modeling: both models, in all four configurations, reconstruct an empty candidate set from the scheduler traces and fall through to a degenerate one-variable model that “solves” successfully, which is a failure mode that objective-only scoring at toy scale would never surface. That diagnosis stands, but the zero no longer does: the statement revision of Section 4.1 landed after this sweep was archived, and on the current corpus the problem passes 5/5 on every repeat (Section 6.8). We leave the row as measured rather than back-fill it, because Table 3 reports one sweep over one frozen release and silently updating cells from a different corpus is how a benchmark stops being auditable. Because failures cluster by problem rather than by instance, the effective sample size of the sweep is closer to 17 than to 128, and the harness exposes neither temperature nor seed, so a difference of a few points between two cells of Table 3 should not be read as a ranking. The per-problem matrix, not the aggregate, is the primary result, and the repeated-run row of Section 6.9 is what would give the aggregate error bars. Execution, finally, is not the bottleneck. Semantics are. Solve time is negligible for 13 of 17 problems (sub-second to seconds) and the two problems that do saturate the 900 s budget are also hard for the reference pipeline. The discriminating failures are wrong objectives, missing constraint families, and broken data binding, consistent with observations on reverse-constructed industrial corpora [22] and with the failure taxonomy of solver-in-the-loop training [29].

## 6.4 Where does the competence live?

The result above belongs to a *stack*, not to a model: `claude-opus-5`, plus a cloud coding agent that can execute, read tracebacks, and revise, plus Pyomo and an open solver, plus a closed-book prompt, plus, in one column, an expert-written annotation file. Every one of those layers is a candidate explanation for the gap between 85.0% here and the 26.9% that nine bare LLMs average on reverse-constructed industrial instances of comparable scale, where five fine-tuned specialists score at or below 5.5% [22]. One layer already has direct evidence attached to it: that corpus reports an execution-level failure rate of 47.6%, and its authors state that these failures largely explain the collapse, since most attempts fail before semantic correctness can be assessed. An agent loop that executes, reads the traceback and revises removes that failure class almost by construction, so a substantial part of the gap is plausibly harness rather than model, which is a hypothesis with a measurement attached, not a concession. Table 3 cannot separate the remaining layers, because both of its systems are harnessed coding models.

We therefore regard the attribution, that is, how much of 85.0% is the model, how much the agent loop, and how much the annotation file, as the central open question this benchmark exists to answer, rather than as a supplementary ablation. IndusOPT is constructed so that the question is answerable: the annotation column already isolates one layer, and the bare-LLM and agentic rows of Section 6.9 isolate the others under an unchanged judge. Those measurements are not in this paper, and Section 6.9 says what stands in the way of each. If the harness turns out to carry most of the gap, the practical conclusion for industrial deployment is about system architecture, not model choice, and that conclusion is only reachable on a corpus where the ceiling is high enough for the layers to be distinguishable at all. A benchmark on which everything scores near zero cannot decompose anything.

## 6.5 The annotation probe

Because `annotations.md` isolates exactly the implicit knowledge an engineer would add to a bare requirement, toggling it is a controlled probe of how much each problem’s difficulty lives in unstated constraints. The outcome is fourfold, and model-dependent (Figure 10):

- **Decisive, with one retraction.** On the datacom load-balancing problem both models went from 8/8 with annotations to 0/8 without, which we originally read as the cleanest evidence in the benchmark that a documented implicit constraint (an epigraph variable for peak congestion) is genuinely necessary

information. It does not survive re-running: on the repaired statement the without-annotations condition is unstable rather than zero, and Section 6.8 withdraws the claim. Link protection remains decisive for one model (5/5 vs. 2/5) and indifferent for the other.

- **Reversed.** On the virtual-power-plant problem one model scores 0/15 *with* annotations and 12/15 without: the annotation’s loose linearization of demand-response bounds led it astray, while the other model absorbed the same text harmlessly (15/15). Annotation quality is not model-independent.
- **Indifferent.** Seven problems shift by at most one instance.
- **Degenerate.** One problem is zero in all four cells, so the probe carries no signal there.

The practical lesson for benchmark builders: an ablation gate like this is the only way to know whether an annotation artifact encodes necessary information, redundant information, or misinformation, and the answer can differ per model.

## 6.6 What a zero means: a failure taxonomy

Aggregating over the sweep’s failed instance-runs, “0” has five distinct anatomies, each with different implications:

1. **Construction-time crashes**, from Pyomo API misuse (e.g. building a variable over a Cartesian product of index sets), so the model never exists. Three of the four all-zero *cells* in Table 3 are this, not modeling failures.
2. **Degenerate-model fallbacks**, in which generated code silently substitutes an empty model when its data parsing yields nothing. It counts as “executable and solved”, which is why execution-rate metrics must be read jointly with model size.
3. **Overconstrained models**, infeasible on every instance, and in one case the generated model was an order of magnitude larger than the reference.
4. **Solver capitulation**, meaning correct-looking MINLP models that no available solver cracks within budget, where the judge records these separately rather than blaming the model.
5. **Judge error**, from the repaired cross-check direction of Section 5.3, now zero by construction but kept in the taxonomy because benchmarks that do not audit their judge cannot distinguish this bucket from the others.

Bucket (5) deserves emphasis in both directions. Re-judging the archived sweep also showed the cross-check layer’s *reach*: across 512 instance-runs it actively confirmed feasibility on 42, abstained on 401 because the variable spaces differ, never ran on 69 unjudgeable ones, and caught 0 true violations in this sweep. The layer earns its keep on the problems whose variable indexing follows the data entities directly, and we state plainly that where it abstains, a pass certifies objective agreement, feasibility of the produced solution, and nothing stronger. Constraint coverage (Section 5.4) is the backstop for exactly those cases.

## 6.7 The benchmark measures itself

Two mechanisms make IndusOPT self-correcting in ways we can document rather than assert. First, the drop-one audit now covers 12 of 17 problems and 51 constraint families (Figure 11, with the complete per-family record in Table 6). It finds 1 family with zero coverage and 4 more binding on fewer than half their instances. All are published as known blind spots, and instance redesign to make them binding is tracked in the repository.

The audit earns its place by disagreeing with the results table. Three problems, both sparse-pivoting problems and memory eviction, pass all four configurations of Table 3 perfectly, and a perfect score admits two readings:

the models are right, or the instances cannot tell. Auditing them separates the two, and the answer is not uniform. LU pivot reordering (C2 on 5 of 6) and memory eviction (weakest family on 4 of 8) come back sound, so their perfect scores mean what they appear to. LDL symmetric pivoting does not: its  $C_2$ , the block-strength condition that admits a  $2 \times 2$  pivot, changes the optimum on exactly one of six instances. A model that dropped that condition would score 6/6 on five of them. We would not have known this from the pass rates, which is the entire argument for measuring coverage separately from performance.

Second, evaluation improved the benchmark itself: on the cross-pod load-balancing problem, fixing a generated model’s discrete topology inside the *reference* model produced solutions up to 8.3% better than three archived heuristic references, which were then replaced under the audit trail (with warm-started re-solves guaranteeing no regression). A benchmark whose answer key can absorb improvements without breaking its provenance chain ages better than one frozen at release.

## 6.8 Repeated runs, and what they cost the story

Every cell of Table 3 is one observation. To find out how much that matters we re-ran the primary condition, which is `claude-opus-5` without annotations, 3 times on the 12 problems whose instances solve fast enough to repeat, and we report what it did to our own claims rather than only to the error bars.

**Most cells are stable and a quarter are not.** 9 of 12 problems returned an identical pass count on all 3 runs. 3 did not, and the spread is not cosmetic: memory eviction returned 8/8, 8/8, 4/8, and datacom load balancing returned 0/8, 8/8, 8/8, which is the full range of the scale, from one prompt, on one problem, with one model. A single run of that cell could have reported either a perfect score or a total failure, and Table 3 reports exactly one run of each cell. The practical consequence is the one we had already stated but could not previously quantify: differences of a few points between cells carry no information, and a per-problem zero should be checked before it is interpreted.

**Two of our own per-problem narratives do not survive this.** They are worth spelling out, because both were load-bearing. Table 3 records `CriticalThreadOpt` as zero in all four configurations, and Section 6.6 reads that as a world-modeling failure. On the current corpus it passes 5/5 on all three repeats. The reason is not run variance but chronology: the statement revision of Section 4.1, which supplied the entity conventions and the tie-breaking constant, was merged after the sweep archived in Table 3. The failure was real and its diagnosis was right, but the problem no longer exhibits it.

The second is more uncomfortable. We described datacom load balancing as the benchmark’s cleanest annotation probe, on the strength of both models going 8/8 with annotations to 0/8 without. On the revised statement the without-annotations condition returns 0/8, 8/8, 8/8. Part of that shift is the statement repair, since Section 4.1 reports the same problem moving from 0/8 to 8/8 once the prose named the importance weight, and notes that the ablation had until then been reading a statement defect as evidence about annotations. What the repeats add is that the residual is unstable rather than clean. We are therefore withdrawing the strong form of that claim: on the current corpus we cannot show that this problem’s annotations carry necessary information, and one run in three still fails without them for reasons the single-run design cannot separate from noise. The annotation probe remains a sound instrument, since it is what exposed the defect in the first place, but the specific 8/8-to-0/8 result belongs to a corpus we have since repaired.

**Infrastructure is part of the measurement too.** One of the 36 planned runs never completed: the third link-protection run exhausted its retries on network errors, so that cell has two observations rather than three. Of the two, one scored 5/5 and the other 4/5, failing a single instance on a Pyomo API misuse, namely a constraint rule returning a bare Boolean, which is bucket (1) of the taxonomy above appearing in the wild rather than in an archive.

**A language pilot, and why it settles nothing yet.** IndusOPT’s statements are Chinese, which we have called a realism feature and an unquantified confound. We translated 3 problems into an English parallel corpus, keeping the same formulations, the same data and the same reference solutions and changing only

the prose, and ran the same condition three times on each. Chinese and English agree on all three: 8/8 versus 8/8 on JPEG, 6/6 versus 6/6 on LU pivot reordering, and on memory eviction the English runs are 8/8, 8/8, 8/8 against Chinese 8/8, 8/8, 4/8. It would be a mistake to read that last pair as English being easier. The Chinese variance on that problem is the same variance reported above, and with three runs per language the noise is larger than any language effect this design could detect. Two further caveats bound it: the translation is machine-drafted and reviewed rather than authored by a native domain expert, so a measured gap would confound comprehension with translation fidelity, and three problems is not a sample. What the pilot establishes is that the parallel corpus is usable and the comparison is now runnable, not that language does not matter.

## 6.9 What we have not run, and what it would take

The study above covers one cell of the space the surrounding literature treats as standard: closed-book frontier coding models, with and without annotations. We state what is missing rather than implying coverage, because two of the gaps bear directly on how Table 3 should be read.

**A harness-free control is the one experiment the paper most needs and does not have.** Section 6.4 argues that the gap between our rate and the bare-LLM rates on reverse-constructed corpora is partly the agent loop rather than the model. Establishing how much would require running the same prompts through a plain single-completion interface with no execution, and our evaluation runs through an agent SDK that has no such path: every call is an agent with a shell. That is an infrastructure limitation, not a judgement about what matters, and it is why the attribution of Section 6.4 is posed as the benchmark’s open question rather than answered here. Nothing in the protocol blocks it: any system emitting a `Pyomo build(data)` is judged by the same pinned pipeline.

**A commercial-solver replay would move the conservative denominator from a bound to a measurement.** The 15 unjudgeable instances are unjudgeable because open solvers could not prove their references optimal (Appendix B), and they are the largest problems in the corpus. We expect admitting them to *lower* the reported rate.

Two further gaps are worth naming. Fine-tuned optimization specialists [7, 16, 19] are unmeasured here, and the obstacle is partly a contract mismatch, in that they emit `coptpy` or `gurobipy` against a self-contained prompt rather than `Pyomo` against a data directory, which a translation layer would confound. And human baselines, which ORLM reports on IndustryOR, would say whether the residual failures are hard for people too.

**A measured obstacle: the inlining ceiling.** One barrier to the specialist comparison is not a matter of effort. Every released specialist takes a single self-contained prompt with no side channel for a data file, so running one means serialising each instance’s data directory into the prompt. Measuring that budget over IndusOPT gives a median of 13.5 KB per instance but a maximum of 23 MB: at a conservative 3.5 bytes per token, 21 of 128 instances (16%) exceed a 128k-token context and 11 (9%) exceed one million tokens. The restriction is itself a finding rather than a nuisance: the self-contained prompt that the specialist literature assumes has a hard industrial-scale ceiling, independent of model quality, and no amount of fine-tuning moves it.

## 7 Limitations

We list what the benchmark and the study do not yet establish, in the same spirit of auditability as the rest of the paper.

**Answer-key coverage.** 15 of 128 instances have reference solutions not proven optimal within our audit budget and are excluded from judging, and they skew hard, so pass rates are optimistic for exactly the regime

the benchmark exists to probe. Closing them requires stronger (likely commercial) solvers or longer audit budgets.

**Instance discriminative power.** Over the 12 of 17 problems audited, the drop-one metric found 1 constraint family with zero coverage and 4 binding on fewer than half their instances. Where coverage is weak and the cross-check abstains, a pass certifies objective agreement and solution feasibility only.

**The coverage metric does not scale.** The 5 unaudited problems are unaudited for one reason: drop-one costs one re-solve per family per instance, so it is affordable exactly where instances are small and unaffordable exactly where they are large. A single cross-pod instance takes up to 31 minutes, the MINLP problem does not reliably finish even one baseline solve, and the largest microgrid instance exhausts memory. We report this as a limitation of the metric we are proposing, not only of this corpus: a benchmark’s discriminative power is hardest to certify in the regime where industrial benchmarks most need certifying. Cheaper surrogates, such as dual-value inspection or dropping a family only on the smallest instance of each problem, would trade rigor for reach and we have not evaluated them.

**Single-observation cells.** The evaluation harness exposes neither temperature nor seed, so each cell of Table 3 is one run. The repeated-run study of Section 6.8 bounds what that costs on the tractable half of the corpus, where 3 of 12 problems move between runs and one moves across the full range, but it covers a single model in a single condition, and the five largest problems are not repeated at all, which is where instability would matter most and cost most to measure. Proper confidence intervals need more runs than we have.

**Data origin.** 13 of 17 problems use engineer-authored synthetic data over real scenario structure, 2 are hybrid and 2 carry real measured data, and of those four only the HarmonyOS scheduler traces are *our* production measurements, since the others are public third-party collections (SuiteSparse, Kodak). The realism claim rests on scenario provenance and authorship, not raw measurements, and desensitization (rescaling, anonymization) may remove numerical pathologies present in production data.

**Solver stack.** All auditing and evaluation uses open-source solvers (HiGHS, SCIP, Ipopt, Couenne). The MINLP problem shows that solver capability, not modeling ability, can bound what is judgeable, and commercial-solver replays would tighten both the answer key and bucket (4) of the failure taxonomy.

**Unresolved attribution.** Both evaluated systems are frontier coding models inside the same execution-capable agent harness, so this study cannot apportion the reported pass rate between model capability, the agent loop, and the supplied annotations, and only the last is isolated here. We treat that decomposition as the benchmark’s principal open question rather than a settled result (Section 6.4).

**Ethics, licensing, and data subjects.** The corpus contains no personal data. The one artifact derived from a running production system is the anonymized HarmonyOS `ftrace` scheduler log, which records thread identifiers, wake-up events and timings from a device under test rather than from a user’s device, with thread and process names replaced by opaque identifiers and the mapping withheld. The remaining measured inputs are third-party public collections used under their own terms, namely SuiteSparse matrices for the two pivoting problems and the Kodak image suite behind the JPEG rate–distortion tables, and they are redistributed unmodified with their provenance recorded per problem. Where scenarios touch commercially sensitive quantities, desensitization is applied and declared in the registration card rather than performed silently: site names become opaque identifiers, cost vectors are rescaled by an undisclosed factor chosen to preserve the optimizer’s structure. That rescaling is a real limitation as well as a safeguard, since it can remove numerical pathologies a production instance would have. Two problems carry `internal` sensitivity and their public release follows the same review as the rest (Section 5.6). The risk we consider most plausible is not disclosure but misuse of the benchmark’s own scores: a pass on a problem whose coverage is weak certifies less than it appears to, which is why the coverage record ships with the data rather than living only here.

**Scope of the study.** Two models, one prompting scheme, single-shot: we evaluate closed-book one-pass modeling deliberately, but this leaves multi-turn agentic systems [1, 29] and fine-tuned specialists [16, 19] unmeasured on IndusOPT, and running them is the natural next experiment. The Chinese-language statements are a feature for realism and a confound for models with weaker Chinese comprehension, and we do not yet

quantify that effect beyond the 3-problem pilot of Section 6.8, whose translation is machine-drafted rather than authored, and whose sample is far too small to separate a language effect from the run-to-run noise the same section measures.

## 8 Conclusion

IndusOPT treats industrial NL-to-optimization evaluation as an engineering artifact in its own right: problems written forward from real production scenarios by their owning engineers, instances that scale four orders of magnitude beyond textbook corpora, and a judge that is itself versioned, audited, and, on one occasion, caught being wrong and repaired against archived evidence. The strongest configuration passes 85.0% of judgeable instance-runs, and we release the benchmark because of what that number can be decomposed into rather than in spite of its height. It is measured on the subset that is easy to verify (75.0% on all 128 instances). Part of it is bought with an expert annotation file that first contact with a requirement does not include, worth up to a whole problem where it bites. And most of its passes rest on objective agreement rather than proven structural equivalence, so that counting only the passes our cross-check actually confirmed, the rate is 5.3%, on a corpus where one constraint family is provably undetectable. The per-problem matrix, the annotation probe, and the failure taxonomy then say *where* the task is unsolved: implicit domain constraints, world-modeling from raw traces, MINLP, and large-scale data binding, and 21 instances lie past the point where a self-contained prompt can carry the data at all. The question the benchmark is built to answer next is how that score divides between the model, the agent loop around it, and the domain knowledge handed to it, and a corpus that saturates or collapses can pose no such question.

The benchmark’s forward construction is complementary to reverse-built corpora such as MIPLIB-NL, and the two ideas we think generalize past our implementation, which are cross-validating a candidate solution against the reference model in both directions and measuring whether an instance can detect an omitted constraint at all, are available to any benchmark that stores full solutions rather than scalar answers. We release the corpus for people building optimization-modeling systems to measure themselves against, while the verification and evaluation toolchain stays with us (Section 5.5), so what we publish alongside the problems is the evidence, meaning per-problem coverage, the audited answer keys, and the blind spots we found in our own data, rather than the machinery. The claim we want held to is not that our pipeline is reproducible but that our numbers are falsifiable: every one of them is attached to a problem anyone can now inspect.

## References

- [1] Ali AhmadiTeshnizi, Wenzhi Gao, Herman Brunborg, Shayan Talaei, Connor Lawless, and Madeleine Udell. OptiMUS-0.3: Using large language models to model and solve optimization problems at scale. *arXiv preprint arXiv:2407.19633*, 2024.
- [2] Ali AhmadiTeshnizi, Wenzhi Gao, and Madeleine Udell. OptiMUS: Scalable optimization modeling with (MI)LP solvers and large language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- [3] Laszlo A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966.
- [4] Pietro Belotti, Jon Lee, Leo Liberti, François Margot, and Andreas Wächter. Branching and bounds tightening techniques for non-convex MINLP. *Optimization Methods and Software*, 24(4-5):597–634, 2009.
- [5] Suresh Bolusani et al. The SCIP optimization suite 9.0. Technical report, Zuse Institute Berlin, 2024.

- [6] James R. Bunch and Linda Kaufman. Some stable methods for calculating inertia and solving symmetric linear systems. *Mathematics of Computation*, 31(137):163–179, 1977.
- [7] Yitian Chen, Jingfan Xia, Siyu Shao, Dongdong Ge, and Yinyu Ye. Solver-informed RL: Grounding large language models for authentic optimization modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2505.11792.
- [8] Yitian Chen, Cheng Cheng, Yinan Sun, Zi Ling, and Dongdong Ge. OPT-Engine: Benchmarking the limits of LLMs in optimization modeling via complexity scaling. *arXiv preprint arXiv:2601.19924*, 2026.
- [9] Timothy A. Davis and Yifan Hu. The University of Florida sparse matrix collection. *ACM Transactions on Mathematical Software*, 38(1):1–25, 2011.
- [10] Zezhen Ding, Zhen Tan, Jiheng Zhang, and Tianlong Chen. OR-R1: Automating modeling and solving of operations research optimization problems via test-time reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026. arXiv:2511.09092.
- [11] Iain S. Duff and Jacko Koster. On algorithms for permuting large entries to the diagonal of a sparse matrix. *SIAM Journal on Matrix Analysis and Applications*, 22(4):973–996, 2001.
- [12] Yongchang Fu, Xinjie Huang, Chengjun Dai, Chengzhe Feng, Junshao Zhang, and Hong Zhu. Opti-Agent-Bench: Benchmarking end-to-end optimization R&D agents on real-world business problems. *arXiv preprint arXiv:2607.10768*, 2026.
- [13] Ambros Gleixner, Gregor Hendel, Gerald Gamrath, Tobias Achterberg, Michael Bastubbe, Timo Berthold, Philipp Christophel, Kati Jarck, Thorsten Koch, Jeff Linderoth, Marco Lübbecke, Hans D. Mittelmann, Derya Ozyurt, Ted K. Ralphs, Domenico Salvagnin, and Yuji Shinano. MIPLIB 2017: Data-driven compilation of the 6th mixed-integer programming library. *Mathematical Programming Computation*, 13:443–490, 2021.
- [14] Matthias Grundmann, Vivek Kwatra, and Irfan Essa. Auto-directed video stabilization with robust L1 optimal camera paths. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2011.
- [15] William E. Hart, Jean-Paul Watson, and David L. Woodruff. Pyomo: Modeling and solving mathematical programs in Python. *Mathematical Programming Computation*, 3(3):219–260, 2011.
- [16] Chenyu Huang, Zhengyang Tang, Shixi Hu, Ruoqing Jiang, Xin Zheng, Dongdong Ge, Benyou Wang, and Zizhuo Wang. ORLM: A customizable framework in training large models for automated optimization modeling. *Operations Research*, 2025. arXiv:2405.17743.
- [17] Xuhan Huang, Qingning Shen, Yan Hu, Anningzhe Gao, and Benyou Wang. Mamo: A mathematical modeling benchmark with solvers. In *Findings of the Association for Computational Linguistics: NAACL*, 2025. arXiv:2405.13144.
- [18] Qi Huangfu and J. A. Julian Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018.
- [19] Caigao Jiang, Xiang Shu, Hong Qian, Xingyu Lu, Jun Zhou, Aimin Zhou, and Yang Yu. LLMOPT: Learning to define and solve general optimization problems from scratch. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.13213.

- [20] Zhong Li, Zihan Guo, Xiaohan Lu, Juntao Wang, Jie Song, Chao Shen, Jiageng Wu, and Mingyang Sun. OptArgus: A multi-agent system to detect hallucinations in LLM-based optimization modeling. *arXiv preprint arXiv:2605.11738*, 2026.
- [21] Zhong Li, Qi Huang, Yi Zhu, Mohammad Mohammadi Amiri, Niki van Stein, Thomas Bäck, Matthijs van Leeuwen, Zaiwen Wen, and Liang Yang. MM-OptBench: A solver-grounded benchmark for multimodal optimization modeling. *arXiv preprint arXiv:2605.12154*, 2026.
- [22] Zhong Li, Hongliang Lu, Tao Wei, Yuxuan Chen, Wenyu Liu, Yuan Lan, Fan Zhang, and Zaiwen Wen. Constructing industrial-scale optimization modeling benchmark. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. URL <https://openreview.net/forum?id=0Q7adIsv6p>. arXiv:2602.10450.
- [23] Keyu Liang, Yifan Lu, Jian Mao, Shaoning Sun, Chao Yang, Cheng Zeng, Xin Jin, Hu Qin, Ruihao Zhu, and Chung-Piaw Teo. LLM for large-scale optimization model auto-formulation: A lightweight few-shot learning approach. *arXiv preprint arXiv:2601.09635*, 2026.
- [24] Vinicius Lima, Dzung T. Phan, Jayant Kalagnanam, Dhaval Patel, and Nianjun Zhou. Toward a trustworthy optimization modeling agent via verifiable synthetic data generation. *arXiv preprint arXiv:2508.03117*, 2025.
- [25] Fan Liu, Zherui Yang, Cancheng Liu, Tianrui Song, Xiaofeng Gao, and Hao Liu. MM-Agent: LLM as agents for real-world mathematical modeling problems. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2505.14148.
- [26] Haoyang Liu, Jie Wang, Yuyang Cai, Xiongwei Han, Yufei Kuang, and Jianye Hao. OptiTree: Hierarchical thoughts generation with tree search for LLM optimization modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2510.22192.
- [27] Wenyu Liu, Hao Wu, Yufei Kuang, Xiongwei Han, Tao Zhong, Jie Feng, and Wenqi Lu. Automated optimization modeling via a localizable error-driven perspective. *arXiv preprint arXiv:2602.11164*, 2026.
- [28] Hongliang Lu, Zhonglin Xie, Yaoyu Wu, Can Ren, Yuxuan Chen, and Zaiwen Wen. OptMATH: A scalable bidirectional data synthesis framework for optimization modeling. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. arXiv:2502.11102.
- [29] Hongliang Lu, Yuxuan Chen, Zhong Li, Zaiwen Wen, Yuan Lan, and Fan Zhang. PEARL: Solver-in-the-loop interactive optimization modeling from natural language. *arXiv preprint arXiv:2607.18256*, 2026.
- [30] Mahdi Mostajabdaveh, Timothy T. Yu, Rindranirina Ramamonjison, Giuseppe Carenini, Zirui Zhou, and Yong Zhang. Optimization modeling and verification from problem specifications using a multi-agent multi-stage LLM framework. *INFOR: Information Systems and Operational Research*, 62(4):599–617, 2024.
- [31] Mahdi Mostajabdaveh, Timothy T. Yu, Samarendra Chandan Bindu Dash, Rindranirina Ramamonjison, Jabo Serge Byusa, Giuseppe Carenini, Zirui Zhou, and Yong Zhang. Evaluating LLM reasoning in the operations research domain with ORQA. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025. arXiv:2412.17874.

- [32] Rindranirina Ramamonjison, Timothy T. Yu, Raymond Li, Haley Li, Giuseppe Carenini, Bissan Ghaddar, Shiqi He, Mahdi Mostajabdaveh, Amin Banitalebi-Dehkordi, Zirui Zhou, and Yong Zhang. NL4Opt competition: Formulating optimization problems based on their natural language descriptions. In *Proceedings of the NeurIPS 2022 Competitions Track*, 2023. arXiv:2303.08233.
- [33] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.
- [34] Weichun Shi et al. ConstraintLLM: A neuro-symbolic framework for industrial-level constraint programming. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 16000–16020, 2025. arXiv:2510.05774; introduces the IndusCP benchmark.
- [35] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. arXiv:2303.11366.
- [36] Andreas Wächter and Lorenz T. Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1):25–57, 2006.
- [37] Gregory K. Wallace. The JPEG still picture compression standard. *IEEE Transactions on Consumer Electronics*, 38(1):xviii–xxxiv, 1992.
- [38] Zhuohan Wang, Ziwei Zhu, Ziniu Li, Congliang Chen, Yizhou Han, Yufeng Lin, Zhihang Lin, Angyang Gu, Xinglin Hu, Ruoyu Sun, and Tian Ding. ORGEval: Graph-theoretic evaluation of LLMs in optimization modeling. *arXiv preprint arXiv:2510.27610*, 2025.
- [39] Yang Wu, Yifan Zhang, Yurong Wu, Yuran Wang, Junkai Zhang, and Jian Cheng. Step-Opt: Boosting optimization modeling in LLMs through iterative data synthesis and structured validation. *arXiv preprint arXiv:2506.17637*, 2025.
- [40] Ziyang Xiao, Dongxiang Zhang, Yangjun Wu, Lilin Xu, Yuan Jessica Wang, Xiongwei Han, Xiaojin Fu, Tao Zhong, Jia Zeng, Mingli Song, and Gang Chen. Chain-of-experts: When LLMs meet complex operations research problems. In *International Conference on Learning Representations (ICLR)*, 2024.
- [41] Beino Yang, Qishen Zhou, Junyi Li, Chenxing Su, Panagiotis Angeloudis, and Simon Hu. ORThought: Benchmarking and automating logistics optimization modeling via structured LLM reasoning. *arXiv preprint arXiv:2508.14410*, 2025.
- [42] Zhicheng Yang, Yiwei Wang, Yinya Huang, Zhijiang Guo, Wei Shi, Xiongwei Han, Liang Feng, Linqi Song, Xiaodan Liang, and Jing Tang. OptiBench meets ReSocratic: Measure and improve LLMs for optimization modeling. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2407.09887.
- [43] Hui Zang, Jason P. Jue, and Biswanath Mukherjee. A review of routing and wavelength assignment approaches for wavelength-routed optical WDM networks. *Optical Networks Magazine*, 1(1):47–60, 2000.
- [44] Mingyang Zhang, Jianan Zhang, Rui Wang, Ramesh Govindan, Jeffrey C. Mogul, and Amin Vahdat. Gemini: Practical reconfigurable datacenter networks with topology and traffic engineering. *arXiv preprint arXiv:2110.08374*, 2021.

- [45] Yansen Zhang, Qingcan Kang, Wing Yin Yu, Hailei Gong, Xiaojin Fu, Xiongwei Han, Tao Zhong, and Chen Ma. Decision information meets large language models: The future of explainable operations research. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2502.09994.
- [46] Chenyu Zhou, Tianyi Xu, Jianghao Lin, and Dongdong Ge. StepORLM: A self-evolving framework with generative process supervision for operations research language models. *arXiv preprint arXiv:2509.22558*, 2025.

## A Problem Inventory

Table 4: All 17 problems. Size is #vars + #cons of the reference model, and min and max are over the problem’s instances. “Fam.” is the number of constraint families a submission has to get right, and “Int.” the fraction of variables that are integral, both measured on the problem’s smallest instance. These are structural properties of the formulation, and they separate problems that size alone ranks together.

| Problem                                           | Class | Inst. | Min size | Max size  | Fam. | Int. |
|---------------------------------------------------|-------|-------|----------|-----------|------|------|
| CBG-Camera-JPEGQuantizationTable                  | MILP  | 8     | 1,471    | 2,457     | 3    | 1.00 |
| CBG-Camera-VideoStabilization-L1                  | LP    | 8     | 520      | 1,204     | 4    | 0.00 |
| CBG-Camera-VideoStabilization-L2                  | QP    | 8     | 160      | 358       | 2    | 0.00 |
| CBG-Communication-RailCellHandover                | MILP  | 5     | 34       | 620       | 6    | 0.62 |
| CBG-HarmonyOS-CriticalThreadOpt                   | MILP  | 5     | 1,046    | 1,386     | 3    | 1.00 |
| CBG-HarmonyOS-MemoryEviction                      | MILP  | 8     | 161      | 654       | 4    | 1.00 |
| Compute-CAE-SparseLA-LDLSymmetricPivoting         | IP    | 6     | 264      | 4,054     | 2    | 1.00 |
| Compute-CAE-SparseLA-LUPivotReordering            | IP    | 6     | 415      | 5,994     | 2    | 1.00 |
| Compute-Cluster-CrossPodLoadBalancing             | MILP  | 10    | 301      | 34,145    | 3    | 0.17 |
| Compute-LLM-MoEExpertLoadBalance                  | MINLP | 5     | 204      | 761,018   | 5    | 0.52 |
| Compute-TBE-MemoryAllocation                      | MILP  | 8     | 3,141    | 176,099   | 18   | 1.00 |
| Energy-Microgrid-SizingAndOperation               | MILP  | 10    | 6,012    | 5,310,086 | 13   | 0.29 |
| Energy-VPP-DayAheadAdjustableLoadScheduling       | MILP  | 15    | 5,356    | 1,269,056 | 17   | 0.19 |
| ICT-DataCom-LoadBalancing-SingleAndMultitimestamp | MILP  | 8     | 33       | 677       | 3    | 0.23 |
| ICT-DataCom-NetworkPlanning-CapacityExpansion     | MILP  | 8     | 528      | 7,336     | 4    | 0.31 |
| ICT-OpticalNetwork-NetworkPlanning-LinkProtection | MILP  | 5     | 974      | 14,424    | 10   | 1.00 |
| ICT-OpticalNetwork-NetworkPlanning-PathProtection | MILP  | 5     | 198      | 5,172     | 8    | 1.00 |

Size is the dimension Figure 2 makes visible, but it is not the only one that matters, and the two do not track each other. Constraint-family count ranges from 2 to 18 with a median of 4: the compiler memory-allocation and virtual-power-plant problems ask for 18 and 17 distinct families respectively, so a submission has that many separate opportunities to omit something, while the two sparse-pivoting problems reach thousands of variables with two families each. Integrality splits the corpus rather than grading it, since 8 of 17 problems are purely integral and 2 purely continuous, with the rest genuinely mixed. A benchmark reported only by variable count would rank the pivoting problems alongside the compiler problem, though nothing about modelling them is comparable.

## B Reference Keys, In Full

Which instances are judgeable is not a property of any submission: it is a property of how strong an answer key we were able to prove. Table 5 gives the reference-solution status of every instance in the release. Only

optimal keys make an instance judgeable, while feasible and heuristic are excluded from scoring rather than used as if they were optima, which is the discipline that a benchmark quietly trusting its weakest keys would skip (Section 4.4). The 15 excluded instances are concentrated in four problems, all of them among the largest, which is why the judgeable denominator is biased toward what is easy to verify (Section 6).

Table 5: Reference-solution status per problem. optimal keys are proven under the zero-gap audit, feasible keys carry a recorded bound but no proof of optimality, and heuristic keys are constructed solutions honestly labelled as such. Only the first column contributes judgeable instances.

| Problem                                           | Inst. | opt. | feas. | heur. | Judgeable | Audit      |
|---------------------------------------------------|-------|------|-------|-------|-----------|------------|
| CBG-Camera-JPEGQuantizationTable                  | 8     | 8    | –     | –     | 8/8       | 8 / 0 / 0  |
| CBG-Camera-VideoStabilization-L1                  | 8     | 8    | –     | –     | 8/8       | 8 / 0 / 0  |
| CBG-Camera-VideoStabilization-L2                  | 8     | 8    | –     | –     | 8/8       | 8 / 0 / 0  |
| CBG-Communication-RailCellHandover                | 5     | 5    | –     | –     | 5/5       | 5 / 0 / 0  |
| CBG-HarmonyOS-CriticalThreadOpt                   | 5     | 5    | –     | –     | 5/5       | 5 / 0 / 0  |
| CBG-HarmonyOS-MemoryEviction                      | 8     | 8    | –     | –     | 8/8       | 8 / 0 / 0  |
| Compute-CAE-SparseLA-LDLSymmetricPivoting         | 6     | 6    | –     | –     | 6/6       | 6 / 0 / 0  |
| Compute-CAE-SparseLA-LUPivotReordering            | 6     | 6    | –     | –     | 6/6       | 6 / 0 / 0  |
| Compute-Cluster-CrossPodLoadBalancing             | 10    | 3    | 7     | –     | 3/10      | 2 / 8 / 0  |
| Compute-LLM-MoEExpertLoadBalance                  | 5     | 4    | –     | 1     | 4/5       | 2 / 2 / 0  |
| Compute-TBE-MemoryAllocation                      | 8     | 5    | 3     | –     | 5/8       | 4 / 4 / 0  |
| Energy-Microgrid-SizingAndOperation               | 10    | 6    | 1     | 3     | 6/10      | 6 / 1 / 0  |
| Energy-VPP-DayAheadAdjustableLoadScheduling       | 15    | 15   | –     | –     | 15/15     | 15 / 0 / 0 |
| ICT-DataCom-LoadBalancing-SingleAndMultitimestamp | 8     | 8    | –     | –     | 8/8       | 8 / 0 / 0  |
| ICT-DataCom-NetworkPlanning-CapacityExpansion     | 8     | 8    | –     | –     | 8/8       | 8 / 0 / 0  |
| ICT-OpticalNetwork-NetworkPlanning-LinkProtection | 5     | 5    | –     | –     | 5/5       | 5 / 0 / 0  |
| ICT-OpticalNetwork-NetworkPlanning-PathProtection | 5     | 5    | –     | –     | 5/5       | 5 / 0 / 0  |

The last column is the outcome of the optimality audit re-run for this release (`tools/audit_solution.py`, gap pinned to zero), as consistent / unproven / contradicted. Over all 128 instances it reports 109 consistent, 15 unproven, 0 contradicted, with 4 honestly labelled heuristic and therefore exempt. Zero contradictions is the result we want but it is also the weaker of the two readings available: 15 instances are unproven, meaning the budget expired without either confirming or refuting the key, and Section 4.4 records a case where an audit that reported no contradictions was simply running at too loose a tolerance to see them. Unproven is not evidence of correctness, and we report it as its own column rather than folding it into the good news.

## C Constraint Coverage, In Full

Table 6 is the complete output of the drop-one audit: every constraint family of every audited problem, with the number of instances on which deactivating that family moves the optimum. We publish it whole rather than as the weakest-family summary of Figure 11 because the audit toolchain is not part of the release (Section 5.5), which makes these findings the evidence a reader has to work from. Bold marks a family no instance can detect. The record is also shipped with the corpus as `coverage.json` per problem, so a benchmark consumer can weigh their own score on a problem against what that problem is able to measure.

## D Problem Gallery

One paragraph per problem: the production context, the decision and objective, and what makes it hard. Descriptions stay at the abstraction level of the published statements.

Table 6: Drop-one constraint coverage. Each entry is  $C_n$  followed by the number of instances on which deactivating  $C_n$  changes the optimum, out of the problem’s instance count. **Bold** = zero coverage (blind spot).

| Problem                                            | Coverage per constraint family                                                                                            |
|----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| CBG-Camera-JPEGQuantizationTable                   | C1 6/8, C2 8/8, C3 5/8                                                                                                    |
| CBG-Camera-VideoStabilization-L1                   | C1 2/8, C2 7/8, C3 8/8, C4 8/8                                                                                            |
| CBG-Camera-VideoStabilization-L2                   | C1 2/8, C2 7/8                                                                                                            |
| CBG-Communication-RailCellHandover                 | C1 5/5, C2 5/5, C3 3/5, C4 3/5, C5 3/5, C6 5/5                                                                            |
| CBG-HarmonyOS-CriticalThreadOpt                    | C1 5/5, <b>C2 0/5</b> , C3 5/5                                                                                            |
| CBG-HarmonyOS-MemoryEviction                       | C1 7/8, C2 4/8, C3 8/8, C4 8/8                                                                                            |
| Compute-CAE-SparseLA-LDLSymmetric Pivoting         | C1 6/6, C2 1/6                                                                                                            |
| Compute-CAE-SparseLA-LUPivot Reordering            | C1 6/6, C2 5/6                                                                                                            |
| ICT-DataCom-LoadBalancing-SingleAnd Multitimestamp | C1 8/8, C2 8/8, C3 8/8                                                                                                    |
| ICT-DataCom-NetworkPlanning-Capacity Expansion     | C1 8/8, C2 8/8, C3 8/8, C4 8/8                                                                                            |
| ICT-OpticalNetwork-NetworkPlanning-LinkProtection  | C1 5/5, C2 5/5, C3 1/5, C4 5/5, C5 5/5, C6 5/5, C7 5/5, C8 4/5, C9 5/5, C10 5/5                                           |
| ICT-OpticalNetwork-NetworkPlanning-PathProtection  | C1 5/5, C2 5/5, C3 5/5, C4 5/5, C5 4/5, C6 5/5, C7 5/5, C8 5/5                                                            |
| Compute-Cluster-CrossPodLoad Balancing             | <i>not audited, because drop-one needs one re-solve per family per instance, and a single instance takes up to 31 min</i> |
| Compute-LLM-MoEExpertLoadBalance                   | <i>not audited, because this is an MINLP whose baseline solve alone does not finish reliably within budget</i>            |
| Compute-TBE-MemoryAllocation                       | <i>not audited, being of the same order as Cross-pod, with the largest instances reaching 1.8e5 vars+cons</i>             |
| Energy-Microgrid-SizingAndOperation                | <i>not audited, because the largest instance is 5.3e6 vars+cons and exhausts memory on a single solve</i>                 |
| Energy-VPP-DayAheadAdjustableLoad Scheduling       | <i>not audited, because 17 families x 15 instances of re-solves take hours of wall clock</i>                              |

**CBG-Camera-JPEGQuantizationTable.** A smartphone imaging pipeline ships a single 8×8 JPEG luminance quantization table in firmware, shared by all images rather than tuned per photo. The MILP selects, for each of the 64 DCT frequency positions, one integer step size from a discrete candidate set to minimize total estimated bitrate over representative images, subject to a hard per-image quality floor (each image’s reconstruction PSNR may not fall below its quality-75 baseline, pre-converted to a constant MSE bound) [37]. Rate and distortion are precomputed per-frequency, per-candidate tables, so the model performs pure discrete selection with an additive distortion model, and the easier per-image table-assignment relaxation is explicitly forbidden. Instances are built from the public Kodak image suite and vary the number of representative images, the PSNR slack, and the candidate-grid density.

**CBG-Camera-VideoStabilization-L1.** In handheld video stabilization, a virtual camera trajectory guides a slightly smaller crop window through the raw frames to cancel hand shake while preserving intentional motion. Each instance is one solve of an online sliding window: the variables are the smoothed 3-DOF camera pose of every active frame, and the objective is a weighted  $\ell_1$  sum of four penalty classes (fidelity, windowed drift, first- and second-order smoothness), linearized with auxiliary variables into an LP whose optima favor piecewise-constant, cinematic-pan motion, in the spirit of L1 camera-path stabilization [14]. Hard constraints keep the crop window strictly inside the frame (per-frame one-sided corner bounds derived from camera geometry) and join the solution seamlessly onto anchor frames finalized by the previous window. Instances vary window length (full, tail, cold-start), shake intensity, crop-margin tightness, and motion type, and share byte-identical data with the L2 variant.

**CBG-Camera-VideoStabilization-L2.** The quadratic counterpart: the same online sliding-window trajectory smoothing, with the four penalty classes priced by squared deviations, giving a convex QP whose optima spread unavoidable deviation thinly across frames. The decisions are again per-frame smoothed camera poses, constrained by no-black-border crop bounds and equality anchoring to previously finalized frames, with limited lookahead from the device’s finite frame buffer. The instance set mirrors the L1 version, including a square-to-wide aspect-ratio geometry, isolating the effect of the objective norm on an otherwise identical model.

**CBG-Communication-RailCellHandover.** On high-speed rail and metro lines, terminals traverse many cells in quick succession, and purely signal-driven handover causes ping-pong switching and dead-end “island” cells, so a cloud-side experience map instead pre-provisions candidate handover paths to the modem. Given an origin–destination passenger distribution, the MILP selects which candidate paths to activate at each origin cell, under per-origin budgets on provisioned paths and next hops, and assigns each unsplittable flow to an activated path, minimizing passenger-weighted node and handover-edge experience costs plus congestion-overflow penalties on shared edge capacities. Paths from multiple origins merge into common downstream corridors, so shared capacities couple otherwise separable origin subproblems. Instances span 6–72 cells and stress distinct pathologies: congestion, island risk, ping-pong risk.

**CBG-HarmonyOS-CriticalThreadOpt.** During camera cold start, the main thread depends on a web of helper threads via wakeup chains, and low-priority dependents delay the critical path, so the operator boosts a subset of thread roles to the main thread’s priority under a limited scheduling-intervention budget. The model is a knapsack-style MILP over roles: binaries select which roles to boost, the objective maximizes a critical-dependency score, and each role’s cost is the product of priority-gap and activity tiers. The instance data are real anonymized `ftrace` scheduler logs (`sched_waking/sched_switch`), and much of the difficulty is data-bound: the wakeup forest, critical-path membership, role statistics, and eligibility filters must be reconstructed from raw events by precise rules before the optimization is even posed, which is exactly where all four evaluation configurations failed (Section 6.6).

**CBG-HarmonyOS-MemoryEviction.** A terminal OS maintains a file page cache at 4 KB granularity, and eviction decisions on a full cache determine page-fault latency during app launches and UI switches. Given a recorded access trace and a fixed number of slots, the MILP decides the resident set at each step, and hence when faults occur and which page is evicted, minimizing total faults over an offline replay, providing a clairvoyant bound in the sense of Belady [3] against which online policies are measured. Instances vary trace length, the capacity-to-working-set ratio, and reuse structure (cyclic working sets, scans with revisits, interleaved hotspots).

**Compute-CAE-SparseLA-LDLSymmetricPivoting.** For symmetric indefinite systems factorized as block-diagonal  $LDL^T$ , purely  $1 \times 1$  diagonal pivots stall on near-zero entries, so practical codes pair indices into  $2 \times 2$  pivot blocks while preserving symmetry. This IP computes a static pre-ordering in the spirit of Bunch–Kaufman pivoting [6]: partition the index set into  $1 \times 1$  and  $2 \times 2$  blocks realizable by one symmetric permutation, with  $1 \times 1$  blocks meeting a diagonal-magnitude threshold and  $2 \times 2$  blocks having sufficient block strength, maximizing the product of block strengths (a matching-flavored structure with a log-linearized objective). All matrices are real symmetric instances from the public SuiteSparse collection [9], stored verbatim (order 48–420), with per-instance stability thresholds.

**Compute-CAE-SparseLA-LUPivotReordering.** Before sparse LU factorization, production solvers permute rows and columns to push numerically large entries onto the diagonal. This IP performs the static version

as an assignment problem, the classical large-entries-to-the-diagonal formulation [11]: select  $n$  nonzero entries, no two sharing a row or column, to serve as the permuted diagonal, maximizing the product of their magnitudes while excluding entries below a per-instance threshold, and fill-in and dynamic pivoting are deliberately out of scope. Instances use unmodified SuiteSparse matrices [9] (order 67–1104), so structure is inherited from genuine engineering sparsity patterns.

**Compute-Cluster-CrossPodLoadBalancing.** A compute cluster’s pods may be connected pairwise by identical-capacity full-duplex links, and directed inter-pod demands must be routed directly or via exactly one transit pod. The MILP jointly decides which links to build and how to split each demand across direct and two-hop paths, minimizing the maximum directional link utilization plus pro-rated construction cost, which is a compact abstraction of joint topology and traffic engineering in the spirit of reconfigurable datacenter interconnects [44]. Binary topology decisions interact with the min–max congestion objective under skewed traffic with elephant flows. Instances span 8–64 pods, where small ones are proven optimal while industrial-scale ones are honestly labeled *feasible* with recorded bounds.

**Compute-LLM-MoEExpertLoadBalance.** In Mixture-of-Experts [33] LLM inference with expert parallelism, unevenly hot experts on one accelerator card saturate it and drag whole-layer latency. Given predicted per-layer, per-expert, per-period loads, the model places experts, filling every card’s slots exactly and allowing optional redundancy for replicating hot experts, and, under non-uniform routing, chooses traffic-split ratios across replicas, minimizing the sum over layers and periods of the busiest card’s load. Uniform routing makes load per replica  $1/y$  for  $y$  copies, the bilinear coupling that renders the problem a MINLP, and the placement must stay fixed across all periods. Instances range from didactic to near-production scale (58 layers, 256 experts per layer, 16 cards).

**Compute-TBE-MemoryAllocation.** After an operator-fusion compiler lowers fused operators into one kernel’s IR DAG, the runtime must plan serialized execution order, each action’s execution site and tiling choice, and the movement and residency of intermediate tensors across a memory hierarchy of unlimited-but-slow global memory and small fast on-chip scopes with individual capacities and alignment granularities. The MILP minimizes total execution time, covering compute, transfers priced by raw tensor bytes, and pipeline-switch penalties between DMA, vector, and cube actions, under precedence, per-scope capacity accounting with alignment and double-buffer overheads, and the requirement that final outputs end in global memory. A trivially feasible all-global schedule always exists, making the value of on-chip placement the quantity optimized.

**Energy-Microgrid-SizingAndOperation.** An off-grid microgrid serving a slowly growing campus load is sized and operated over a twenty-year lifecycle: the MILP jointly chooses installed counts of wind turbines, PV modules, diesel generators, and battery units, with batteries deployable in staged batches across years, and schedules dispatch over representative years and periods, minimizing discounted lifecycle cost (investment, O&M, fuel, load-shedding and curtailment penalties). Modeling subtleties include exogenous per-batch battery state-of-health coefficients, PV-only annual degradation, binary charge/discharge exclusion, and per-period SOC closure. The instance set spans a 24-period toy to the benchmark’s largest instance ( $> 5 \times 10^6$  variables plus constraints), the largest carrying heuristic rather than proven-optimal references.

**Energy-VPP-DayAheadAdjustableLoadScheduling.** A virtual power plant aggregating distributed sites submits a day-ahead adjustable load curve over 96 fifteen-minute periods, deciding each site’s grid purchase, storage charge/discharge, and end-of-period energy. Site-level constraints include power balance against forecast load and renewables, efficiency-asymmetric SOC recursion with charge/discharge exclusion, and a

mandatory return to a target end-of-day energy, and the objective minimizes purchase cost less peak-shaving and valley-filling revenues settled at the aggregated VPP level. The key implicit constraint: service response must equal the *positive part* of the deviation between aggregate purchase and the historical baseline in priced periods, which is determined by the schedule rather than being a free decision, ruling out over-reporting. Instances scale from 4 to 1,100 sites, all proven optimal.

**ICT-DataCom-LoadBalancing-SingleAndMultitimestamp.** In a hierarchical enterprise access network, each service at a site is assigned exactly one pre-provisioned path configuration, and within a configuration traffic splits by fixed weights, so a single discrete choice per service determines all link loads. The MILP minimizes a weighted combination of importance-weighted peak utilization of site-attached links, the same measure over upper-tier nodes (each node’s congestion a max over its designated links and over time), and the fraction of services on SLA-violating configurations, under hard link capacities. SLA compliance is judged for a configuration as a whole, so every member path must meet delay, jitter, and loss limits. This problem is the benchmark’s cleanest annotation probe (Section 6.5). Instances vary network size, time periods (1–8), SLA pressure, and capacity headroom.

**ICT-DataCom-NetworkPlanning-CapacityExpansion.** An existing directed backbone/aggregation/edge network carries splittable flows over at most four fixed candidate paths per service, and after a demand change concentrated on hotspot regions, all new demand must be carried. The MILP re-splits each service across its candidate paths and buys integer capacity packs (three sizes with economies of scale), under per-link planning-utilization caps of 70–85% of post-expansion capacity. The operator’s two-level preference, which is to minimize expansion cost first and then traffic migration relative to proportionally scaled original splits, is folded into one objective via an exact lexicographic weight supplied in the data, and every instance is constructed so that a cut around the hotspot forces genuine expansion. Topologies are synthetic but calibrated to public PoP-network statistics (11–52 nodes, up to 440 services).

**ICT-OpticalNetwork-NetworkPlanning-LinkProtection.** Routing and wavelength assignment (RWA) [43] in a WDM network with link protection: for every single-link failure scenario, enumerated over all directed links, backup paths and wavelengths are pre-planned for exactly those demands whose working path traverses the failed link. Decisions comprise working routes and wavelengths plus per-scenario backups under no-wavelength-conversion, with backups forbidden from touching working link-wavelength resources and no wavelength shared on a link in any state, and the objective minimizes total link-wavelength resources touched. A stated implicit constraint prohibits provisioning backups for unaffected demands. The per-scenario copy of the constraint system makes even 4–8-node networks produce MILPs with ~10,000 constraints.

**ICT-OpticalNetwork-NetworkPlanning-PathProtection.** The companion RWA problem under dedicated (1+1) path protection: each demand receives one working path and one link-disjoint backup, each with a single end-to-end wavelength. Backups may not reuse working link-wavelength resources and are not shared between demands, at most one demand occupies a wavelength on a link, and the objective again minimizes occupied link-wavelength pairs to preserve headroom. The single-backup-per-demand structure yields far more compact models than the link-protection variant, making the pair a controlled comparison of two protection architectures on the same planning task.

## E Evaluation Inputs and Solve Regime

This appendix records what the model under test receives and how its output is solved, which are the two things a reader needs in order to judge whether the comparison in Section 6 is fair. The harness that implements

them is not part of the release (Section 5.5), so we describe the contract rather than the code.

**Prompt inputs.** One evaluation call gives the model the problem statement, the data dictionary, and the data files of a *single* sample instance, truncated so as to convey the format while deliberately not being enough to invite hard-coding against values, since the generated code must handle every instance of the problem. In the with-annotations configuration the annotation file is appended. The reference formulation, reference implementation, and all reference solutions are never included, and the sandbox has no repository mount. The prompt is written in Chinese, matching the statements.

**Output contract.** The model must return one Python code block defining `build(data)`, which constructs and returns a Pyomo `ConcreteModel` carrying the decision variables, a single objective, and all constraints. Four requirements make the result judgeable. The function must not solve, because solving is the caller’s job, so that we control the solver. Every number must be read from `data`, with no hard-coded values, so that one submission is tested on all instances. The argument is the parsed instance (a JSON object, or a mapping from file stem to contents, with CSV fields arriving as strings). And only the standard library and Pyomo may be used, so that execution is reproducible on our workers.

**Solve regime.** Solvers rotate by model class, sending LP/MILP to HiGHS [18] then SCIP [5], QP/NLP to Ipopt [36] then Couenne [4], and MINLP to SCIP then Couenne, with 900 s of wall clock per solver, a MIP relative gap and integrality tolerance of  $10^{-6}$ , and a memory cap per worker. A termination of `optimal` with bound equal to objective counts as proven, while a time-limit termination yields an incumbent, judged by the same objective tolerance but recorded together with its bound. The regime and the dataset content are version-pinned together, and a stored result is reused only while both are unchanged, and the sweep reported in this paper was produced under one regime version and re-judged off-line under the repaired cross-check of Section 5.3.

## F Reference-Solution Maintenance

Reference solutions are living artifacts under audit rather than fixed releases. Every reference model is periodically replayed under the evaluation solve regime, warm-started from its recorded solution so that a replay can only match or improve on the stored answer and never regress, and a strictly better incumbent updates the stored objective, bound, and gap under review. The three improved references discussed in Section 6.7 entered the benchmark this way, with the decision log archived. The design point worth carrying over is that an answer key which can absorb an improvement without breaking its provenance chain ages better than one frozen at release, including when the improvement is found by a system under evaluation.

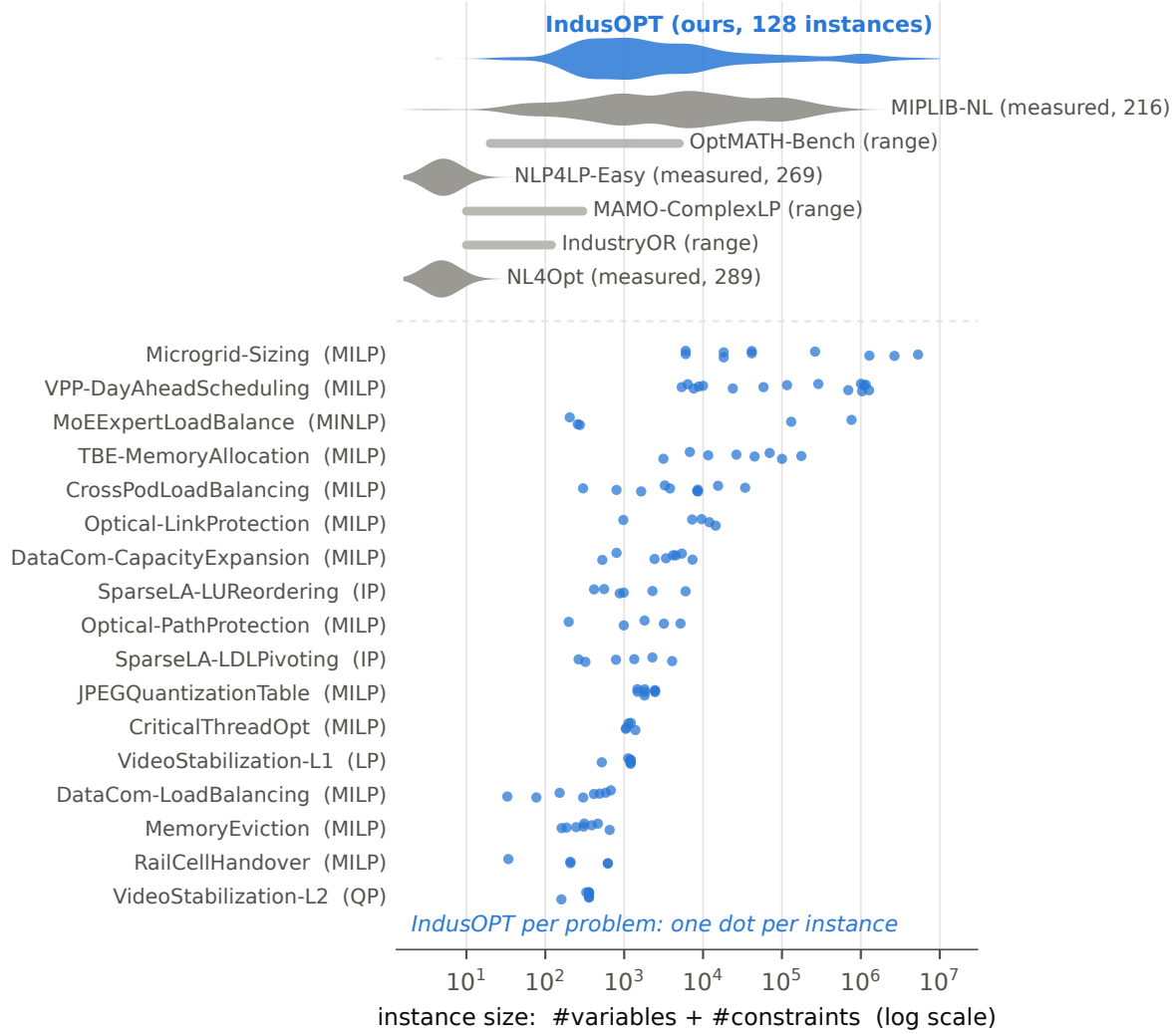


Figure 2: The scale spectrum, as *distributions* rather than ranges (#vars + #cons, log scale). Top: silhouette width is the density of instances at each size. IndusOPT (blue) is exact, from all 128 instances. “Measured” silhouettes we computed ourselves from the benchmarks’ public artifacts (methods and caveats in `paper/bench_sizes.json`): all 289 NL4Opt test problems (median size 5, maximum 8), the 269 publicly mirrored NLP4LP easy problems (median 5, maximum 10), and 216 of MIPLIB-NL’s 223 instances (median 4,879), by executing its published ground-truth build scripts against a counting-only solver shim, digit-exact against MIPLIB 2017 official statistics where comparable. The remaining 7 cannot be built from its released artifacts, owing to a missing shared module, one absent data directory, and one data-code schema mismatch. MAMO, IndustryOR, and OptMATH-Bench release only question text and optimal values, and no reference models exist from which per-instance sizes could be derived, so only surveyed ranges [22] can be drawn. Bottom: one dot per IndusOPT instance, per problem.

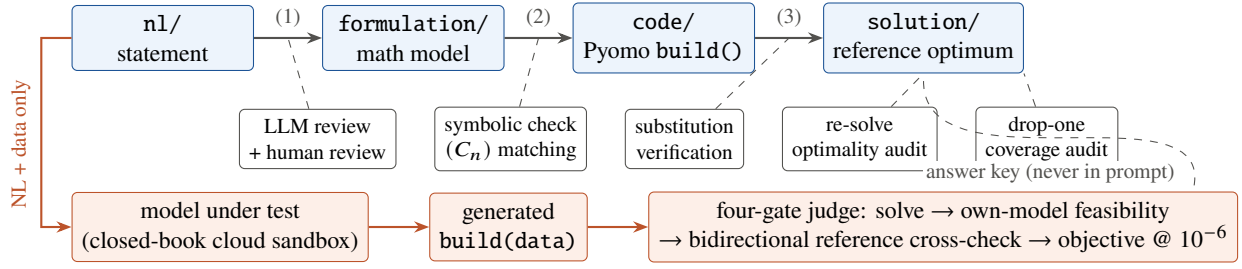


Figure 8: The IndusOPT pipeline. Top (blue): the construction chain, in which every translation between artifacts has its own gate, and the answer key is audited by re-solving and by drop-one-constraint coverage. Bottom (orange): closed-book evaluation, where the model under test sees only the statement and data, and its generated build(data) is executed and judged by us, with the reference solution used only as the answer key.

|                           | composer-2.5 |         | claude-opus-5 |         |
|---------------------------|--------------|---------|---------------|---------|
|                           | with ann     | w/o ann | with ann      | w/o ann |
| SparseLA-LDLPivoting      | 6/6          | 6/6     | 6/6           | 6/6     |
| SparseLA-LURreordering    | 6/6          | 6/6     | 6/6           | 6/6     |
| JPEGQuantizationTable     | 0/8          | 8/8     | 8/8           | 8/8     |
| VideoStabilization-L1     | 8/8          | 6/8     | 8/8           | 8/8     |
| VideoStabilization-L2     | 8/8          | 8/8     | 8/8           | 8/8     |
| RailCellHandover          | 5/5          | 5/5     | 5/5           | 5/5     |
| CriticalThreadOpt         | 0/5          | 0/5     | 0/5           | 0/5     |
| MemoryEviction            | 8/8          | 8/8     | 8/8           | 8/8     |
| CrossPodLoadBalancing     | 3/3          | 3/3     | 3/3           | 0/3     |
| TBE-MemoryAllocation      | 5/5          | 0/5     | 0/5           | 4/5     |
| Microgrid-Sizing          | 6/6          | 6/6     | 0/6           | 6/6     |
| VPP-DayAheadScheduling    | 0/15         | 12/15   | 15/15         | 13/15   |
| DataCom-LoadBalancing     | 8/8          | 0/8     | 8/8           | 0/8     |
| DataCom-CapacityExpansion | 8/8          | 8/8     | 8/8           | 8/8     |
| MoEExpertLoadBalance      | 3/4          | 2/4     | 3/4           | 3/4     |
| Optical-LinkProtection    | 5/5          | 2/5     | 5/5           | 5/5     |
| Optical-PathProtection    | 5/5          | 5/5     | 5/5           | 5/5     |

  all pass
   partial
   zero

Figure 9: Table 3 at a glance: light blue = all judgeable instances pass, dark blue = partial, orange = zero. Failures are concentrated in a few problems and often in specific configurations, not spread uniformly, which is the visual signature of implicit-constraint and world-modeling difficulty rather than random noise.

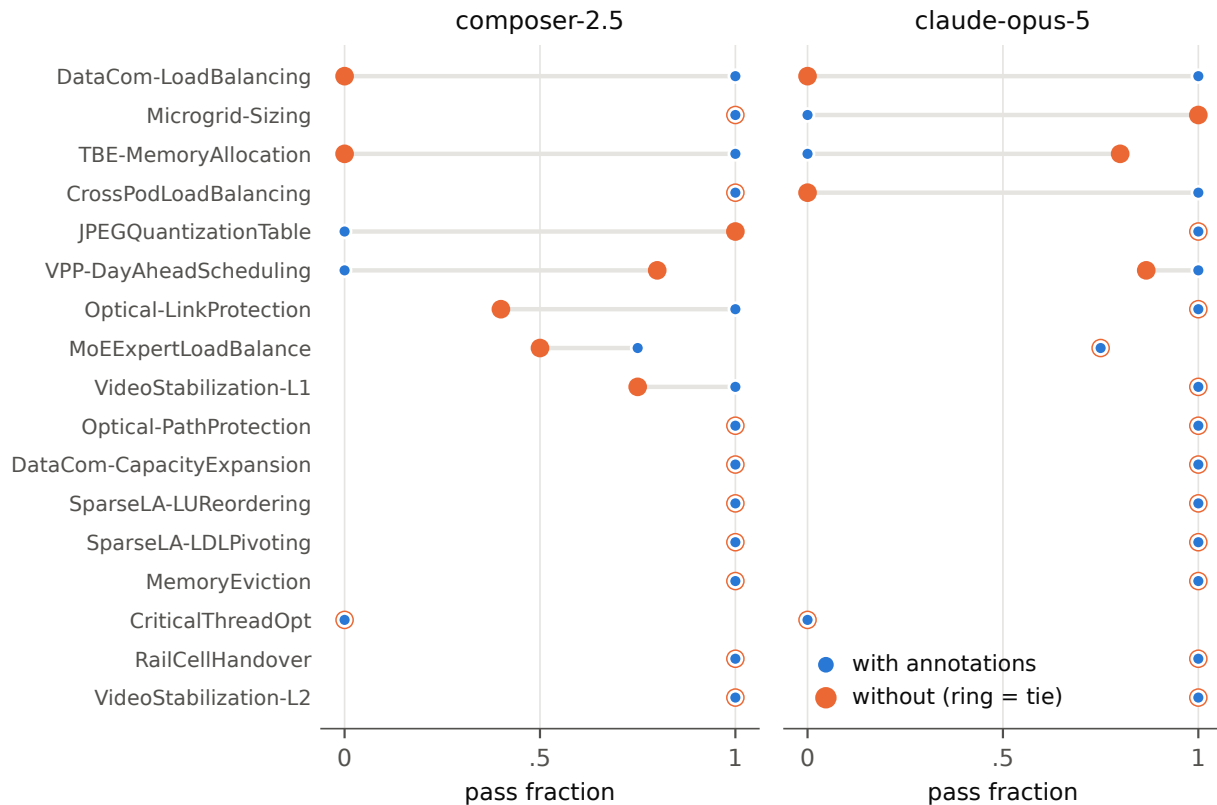


Figure 10: The annotation probe. For each problem and model, pass fraction with annotations (blue) vs. without (orange), and an orange ring around a blue dot marks a tie. Problems are ordered by effect size. The probe separates decisive problems (top) from indifferent ones (bottom), and the same problem can be decisive for one model and indifferent, or even reversed, for the other.

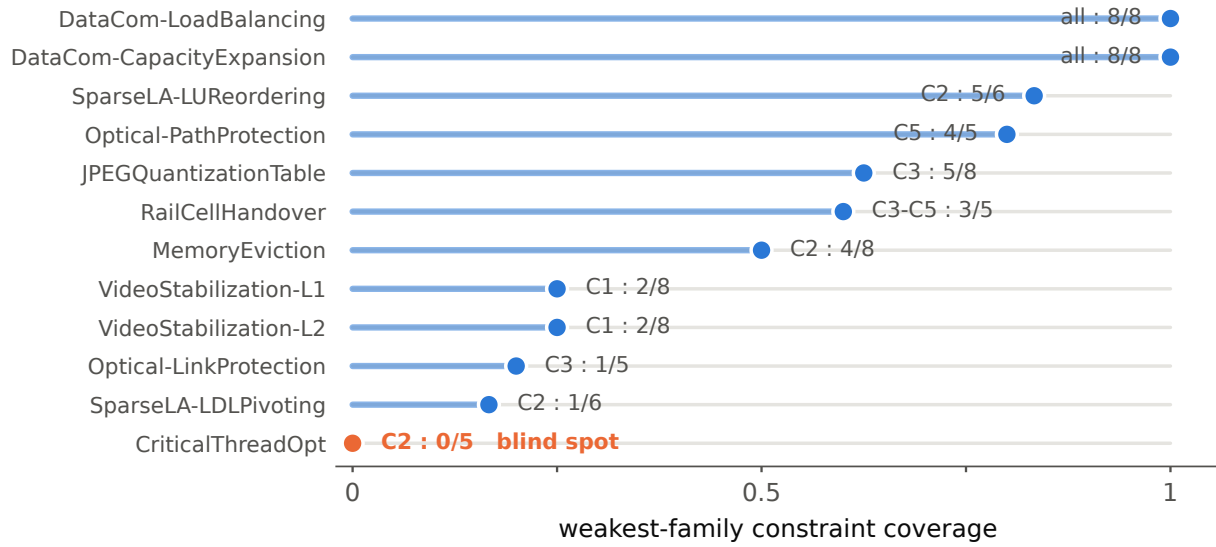


Figure 11: Constraint coverage of the 12 audited problems: for each, the *weakest* constraint family and the fraction of instances on which dropping it changes the optimum. Zero coverage means a model omitting that constraint scores perfectly, which we publish as a blind spot. The complete per-family record, including the 5 problems for which drop-one re-solving is too expensive at their scale, is Table 6.